



CS140 - Introduction To Software Engineering

Week 1

What is Software?

- The product that software professionals build and then support over the long term.
- Software encompasses:
 1. **Instructions/Computer Programs** that when executed provide desired features, functions and performance
 2. **Data Structures** that enables the program to adequately store and manipulate information.
 3. **Documentation** that describes the operation and uses of the program.

Software Products

Generic Products

- Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
- The specification and license remains with developers and user gets access upon paying.
- Example: MS Office, CAD software.

Customized Products

- Software that is commissioned by a **specific customer** to meet their needs.
- The specification and license is owned and controlled by the customer unless otherwise stated in contract.
- Example: Air traffic control system, embedded control systems.

Software Costs

- Software costs more to maintain than it does to develop. For systems with a long life, maintenance cost may be **several times** development costs.

Features of Software?

- Software is **developed and engineered**, it is not manufactured in the classical sense which has quality problems.
- Software doesn't "wear out" but it does deteriorate due to change.
- Has a bathtub

Software Applications

1. **System Software:** compilers, editors, file management utilities.
2. **Application Software:** stand-alone programs for specific needs.
3. **Engineering/Scientific Software:** characterized by "number crunching" algorithms, such as automotive stress analysis, molecular biology, orbital dynamics etc.
4. **Embedded Software:** resides within a product or system, such as key pad control of a microwave oven, digital function of dashboard display in a car
5. **Product-line Software:** focus on a limited marketplace to address mass consumer market, such as word processing, graphics, database management.
6. **WebApps/Web Applications:** network centric software. As web 2.0 emerges, more sophisticated computing environments is supported integrated with remote database and business applications.
7. **AI:** software uses non-numerical algorithms to solve complex problems such as robotics, expert system, pattern recognition game playing.
8. **Open-Source:** "free" source code pen to the computing community. A blessing but also potentially a curse.

IEEE Definition of Software Engineering

- The application of a **systematic, disciplined and quantifiable** approach to the **development, operation and maintenance** of software.

- An engineering discipline that is concerned with all aspects of software production.

Differences Between Sciences and Engineering

- Computer science focuses on **theories and fundamentals**. Software engineering is concerned with the practicalities of developing and delivering useful software.
- System engineering is concerned with **all aspects** of computer-based development including hardware and process engineering. Software engineering is part of this more general process.

Attributes of Good Software

Maintainability

- Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.

Dependability and Security

- Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.

Efficiency

- Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.

Acceptability

- Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

A Layered Technology

Quality Focus

- Any engineering approach must rest on organizational commitment to quality which fosters a continuous process improvement culture.

Process Model

- Process layer as the foundation defines a framework with activities for effective delivery of software engineering technology. Establish the context where products (model, data, report, and forms) are produced, milestone are established, quality is ensured and change is managed.

Methods

- Method provides technical how-to's for building software. It encompasses many tasks including communication, requirement analysis, design modeling, program construction, testing and support.

Tools

- Tools provide automated or semi-automated support for the process and methods.

Software Crisis, Symptoms and Causes

What is Software Crisis?

- Refers to when the complexity of software projects began to exceed the capabilities of existing development methods and tools.
- When the **demand** for software is more than the **production** due to increases in complexities.
- **28%** of projects complete production on time and within budget, **49%** experience delays or exceed budget and the other **23%** of projects are cancelled

Symptoms of Software Crisis & Tracking

- Over budget
 - Can be tracked and monitored using an **S-Curve** to find the variance between predicted and actual costs of production.
- Schedule slippage
 - Can be tracked with **Gantt charts** or **milestone charts** which allocates dates by which each phase of production should be completed.
- Poor quality
 - Can be tracked using **Earned Value Management (EVM)** or **Earned Value Analysis (EVA)** as well as **Customer Satisfaction** and the

absence of bugs or errors, which while isn't a guarantee of quality, shows project is viewed positively and runs competently.

Major Causes of Software Crisis

- Communication difficulties
- Management of complexity

Ethics vs Law

- Ethics are personal code of behavior
- Law is minimum standard imposed by society
- Law represents **will of majority** (violation punishable by government), whereas ethics are **suggested, not mandated** (violation can result in malpractice suit, loss of job)
- Ethics are also referred to as **soft skills** such as honesty and empathy.

Issues of Professional Responsibility

Confidentiality

- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

Competence

- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outside their competence level.

Intellectual Property Rights

- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

Computer Misuse

- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

Code of Ethics

- Express the conscience of the computing profession and it affirms our obligation of computing professionals both individually and collectively to use their skills for the benefit of society.
 - Written by the **ACM (Association for Computing Machinery)**
1. **PUBLIC** - Software engineers shall act consistently with the public interest.
 2. **CLIENT AND EMPLOYER** - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
 3. **PRODUCT** - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
 4. **JUDGMENT** - Software engineers shall maintain integrity and independence in their professional judgment.
 5. **MANAGEMENT** - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
 6. **PROFESSION** - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
 7. **COLLEAGUES** - Software engineers shall be fair to and supportive of their colleagues.
 8. **SELF** - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Week 2

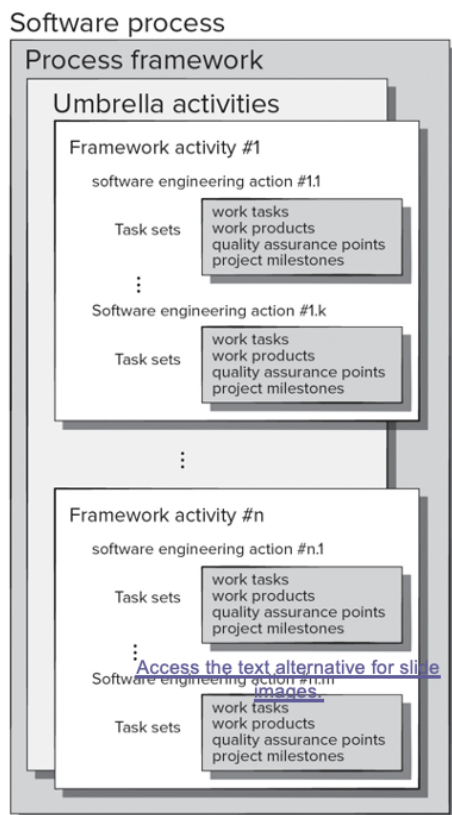
Software Development Life Cycle

- A series of steps, or **phases**, that provide a model for the development and lifecycle management of an application or piece of software.
- Used to ensure good software is built.
- Each phase in the life cycle has its own process and deliverables that feed into the next phase

Why Systems Fail

- Unclear or missing requirements
- Skipping SDLC phases
- Failure to manage project scope
- Failure to manage project plan
- Changing technology

Generic Process Model



Process Framework Activities

- Communication
- Planning
- Modeling
 - Analysis of requirements
 - Design
- Construction
 - Code Generation

- Testing
- Deployment
 - Maintenance

Umbrella Activities

- Software project tracking and control
- Risk management
- Software quality assurance
- Technical reviews
- Measurement
- Software configuration management
- Reusability management
- Work product preparation and production

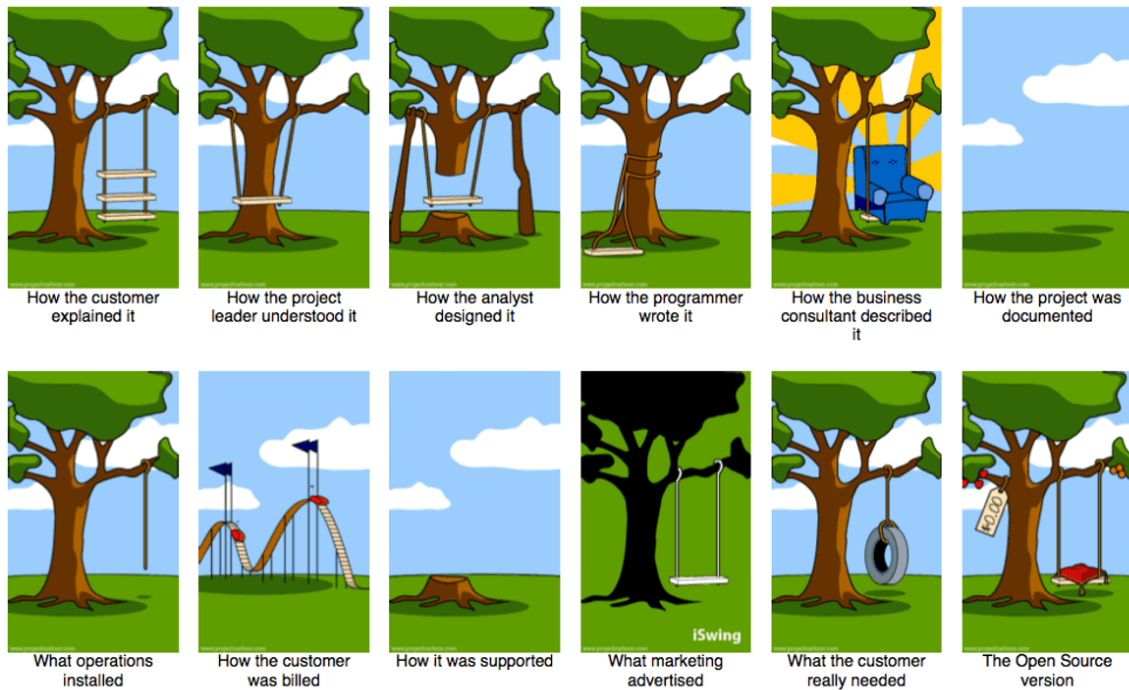
Identifying a Task Set

A task set defines the actual work to be done to accomplish the objectives of a software engineering action.

A task set is defined by creating several lists:

- A list of the tasks to be accomplished.
- A list of the work products to be produced.
- A list of the quality assurance filters to be applied.

The Behavior Effect



Principles that Guide Process

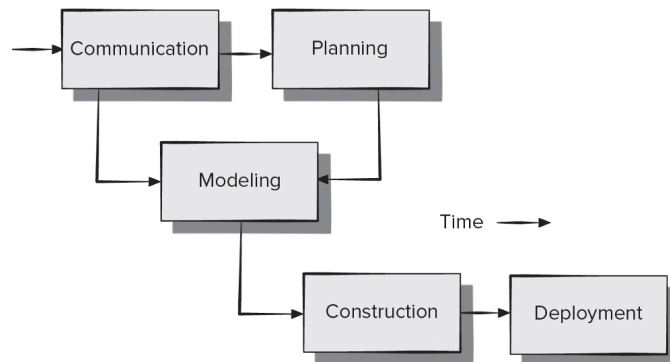
- **Be agile.**
 - Regards of your process model, let the basic tenets of agile development govern your approach.
- **Focus on quality at every step.**
 - The exit condition for every process activity, action, and task should focus on the quality of the work product produced.
- **Be ready to adapt.**
 - Dogma has no place in software development. Adapt your approach to constraints imposed by the problem, the people, and the project itself.
- **Build an effective team.**
 - Software engineering process and practice are important, but the bottom line is people. Build a self-organizing team.
- **Establish mechanisms for communication and coordination.**
 - Projects fail because information falls into the cracks and/or stakeholders fail to coordinate their efforts.
- **Manage change.**

- Approach may formal or informal. You need mechanisms to manage how changes are requested, assessed, approved and implemented.
- **Assess risk.**
 - Lots of things can go wrong as software is being developed, establish contingency plans.
- **Create work products that provide value for others.**
 - Create only those work products that provide value for other process activities, actions or tasks.

Principles that Guide Practice

- **Divide and conquer.**
 - Analysis and design should always emphasize separation of concerns (SoC).
- **Understand the use of abstraction.**
 - Abstraction is a simplification of a complex system element used to communication meaning simply.
- **Strive for consistency.**
 - A familiar context makes software easier to use.
- **Focus on the transfer of information.**
 - Pay special attention to the analysis, design, construction, and testing of interfaces.
- **Build software that exhibits effective modularity.**
 - Provides a mechanism for realizing the philosophy of Separation of concerns .
- **Look for patterns.**
 - The goal of patterns is to create a body of literature to help developers resolve recurring problems encountered in software development.
- **Use multiple viewpoints.**
 - Represent the problem and solution from different perspectives.
- **Some consumes your work products.**
 - Remember that someone will maintain the software.

Simplified Process Model



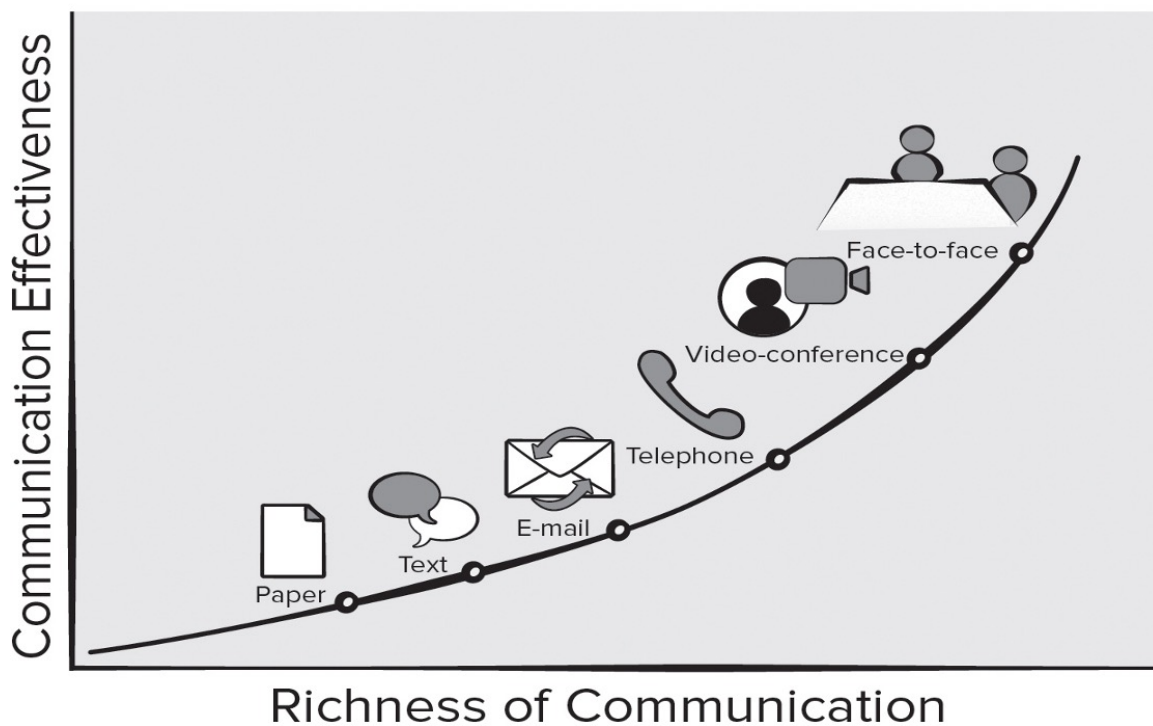
Communication Principles

- **Listen.**
 - Try to focus on the speaker's words, not formulating your response to those words.
- **Prepare before you communicate.**
 - Understand a problem before meeting with others.
- **Someone should facilitate the activity.**
 - Every communication meeting should have a leader to keep the conversation moving in a productive direction.
- **Face-to-face communication is best.**
 - Visual representations of information can be helpful.
- **Take notes and document decisions.**
 - Someone should serve as a "recorder" and write down all important points and decisions.
- **Strive for collaboration.**
 - Consensus occurs when collective team knowledge is combined.
- **Stay focused, modularize your discussion.**
 - The more people involved in communication the more likely discussion will bounce between topics.
- **If something is unclear, draw a picture.**
- **Learn to move on.**

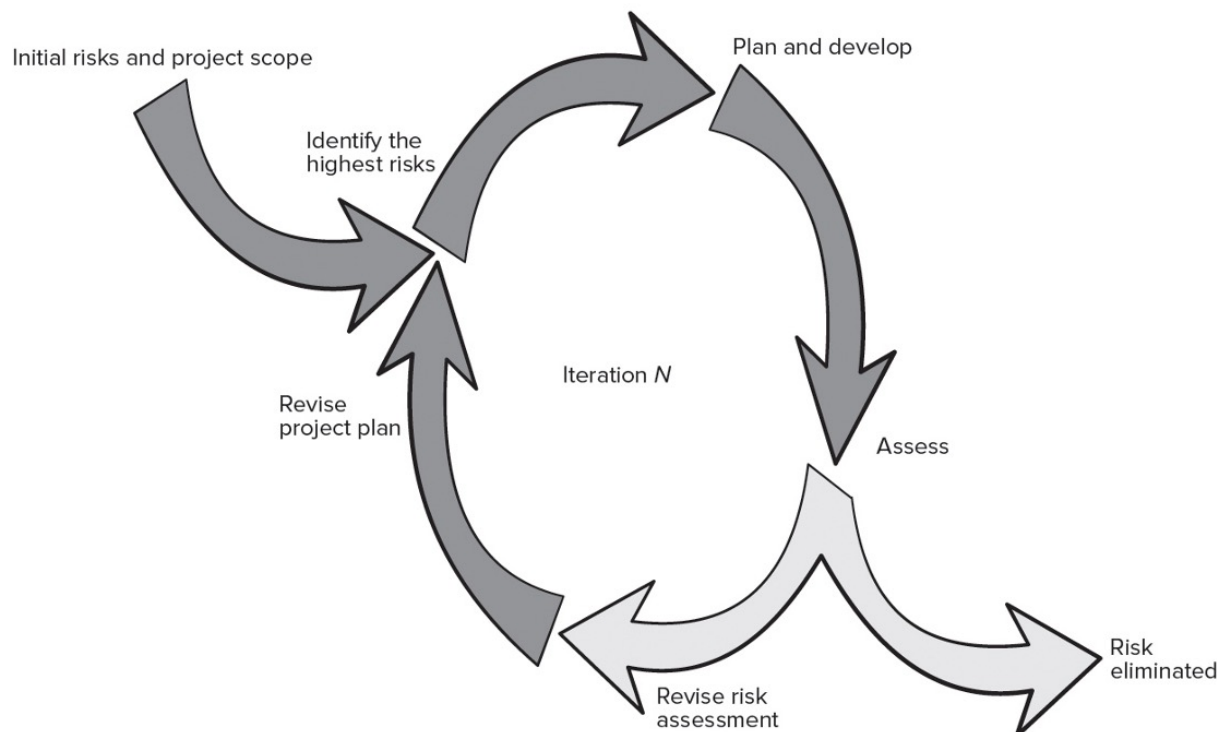
- (a) Once you agree to something, move on;
- (b) If you can't agree to something, move on;
- (c) If a feature or function is unclear and cannot be clarified at the moment, move on.
- **Negotiation is not a contest or a game.**
 - It works best when both parties win.

Communications Mode Effectiveness

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Iterative Planning Process

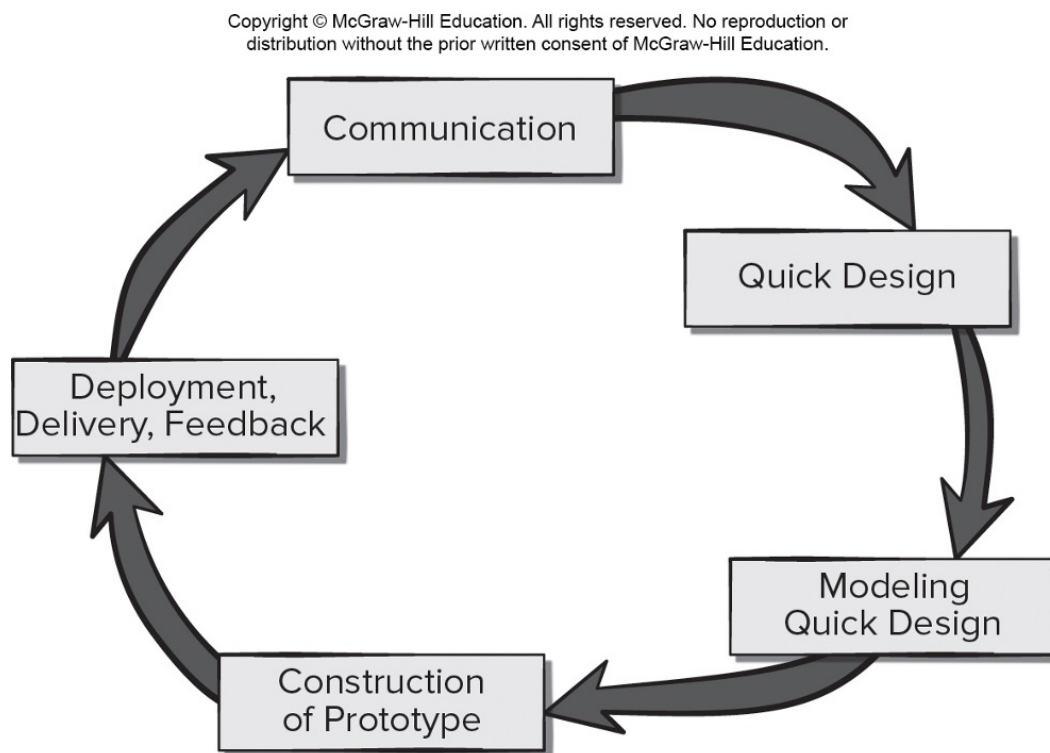


Planning Principles

- **Understand the scope of the project.**
 - Scope provides the software team with a destination as the roadmap is created.
- **Involve the customer in the planning activity.**
 - They define priorities and project constraints.
- **Recognize that planning is iterative.**
 - A project plan is likely to change as work begins.
- **Estimate based on what you know.**
 - Estimation provides an indication of effort, cost, and task duration, based on team's current understanding of work.
- **Consider risk as you define the plan.**
 - Contingency planning is needed for identified high impact and high probability risks.
- **Adjust granularity as you define the plan.**

- Granularity refers to the level of detail that is introduced as a project plan is developed.
- **Define how you intend to ensure quality.**
 - Your plan should identify how the software team intends to ensure quality.
- **Describe how you intend to accommodate change.**
 - Even the best planning can be obviated by uncontrolled change.
- **Track the plan frequently and make adjustments as required.**
 - Software projects fall behind schedule one day at a time.

Software Modeling



Agile Modeling Principles

- **The primary goal of the software team is to build software not create models.**
- **Travel light – don't create more models than you need.**
- **Strive to produce the simplest model that will describe the problem or the software.**

- **Build models in a way that makes them amenable to change.**
- **Be able to state an explicit purpose for each model that is created.**
- **Adapt the models you create to the system at hand.**
- **Try to build useful models, forget about building perfect models.**
- **Don't become dogmatic about model syntax. Successful communication is key.**
- **If your instincts tell you a paper model isn't working you may have a reason to be concerned.**
- **Get feedback as soon as you can.**

Construction Principles - Coding

Preparation Principles: Before you write one line of code, be sure you:

- Understand the problem to be solved.
- Understand basic design principles and concepts.
- Pick a programming language that meets the needs of the software to be built.
- Select a programming environment that provides tools that will make your work easier.
- Create a set of unit tests that will be applied once the component you code is completed.

Coding Principles: As you begin writing code, be sure you:

- Constrain your algorithms by following structured programming practice.
- Consider the use of pair programming.
- Select data structures that will meet the needs of the design.
- Understand the software architecture and create interfaces that are consistent with it.

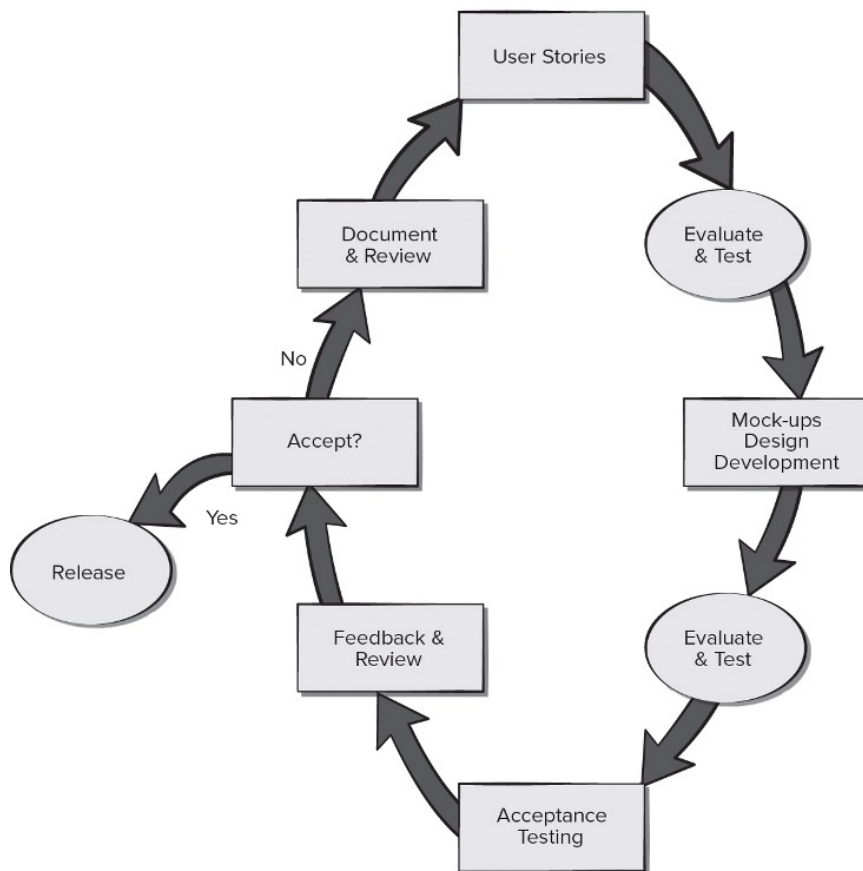
Validation Principles: After you've completed your first coding pass, be sure you:

- Conduct a code walkthrough when appropriate.
- Perform unit tests and correct errors you've uncovered.

- Refactor the code to improve its quality.

Agile Testing

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Testing Principles

- All tests should be **traceable to customer requirements**.
- Tests should be planned long **before testing begins**.
 - Testing is a process of executing a program with **intent of finding an error**,
 - A good test case is one that has a **high probability** of finding an as-yet-undiscovered error.
 - A successful test is one that uncovers an **as-yet-undiscovered error**.
- The **Pareto** principle applies to software testing.
- Testing should begin "in the small" and progress toward testing "in the large."

- Exhaustive testing is not possible.
- Testing effort for each system module commensurate to expected fault density.
- Static testing can yield high results.
- Track defects and look for patterns in defects uncovered by testing.
- Include test cases that demonstrate software is behaving correctly.

Software Deployment Actions

- Assemble deployment package
- Establish Support Regimen
- Manage customer expectations
- Provide instructional materials to end users

Deployment Principles

- Customer expectations for the software must be managed.
- A complete delivery package should be assembled and tested.
- A support regime must be established before the software is delivered.
- Appropriate instructional materials must be provided to end-users.
- Buggy software should be fixed first, delivered later.

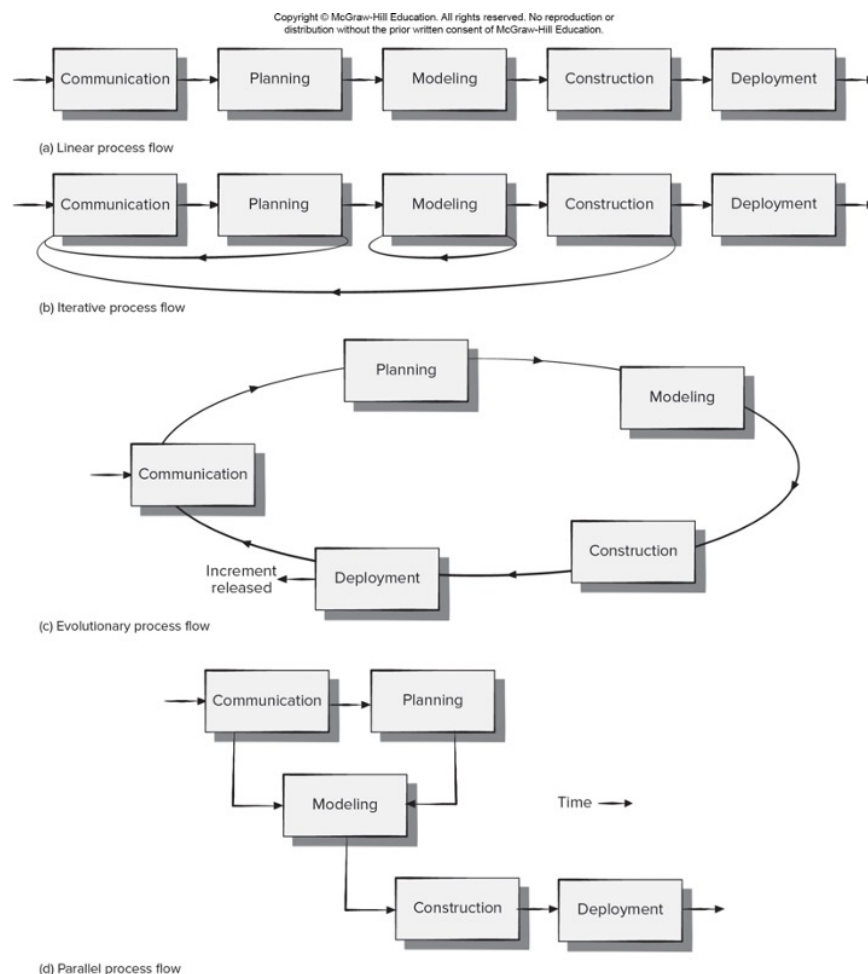
Sourcing

- **Insourcing:**
 - Using IT within the resources of the organization
 - IT specialists within your organization will develop the system.
 - Insourcing is one of the most common methods to develop a system.
 - Typically cheapest
 - Company does not have to hire contractors
- **Selfsourcing:**
 - Using Knowledge Workers

- Self sourcing (also called knowledge worker development or end user development)
- The development and support of IT systems by knowledge workers with little or no help from IT specialists.
- **Outsourcing:**
 - Using another organization
 - The delegation of specific work to a third party for a specified length of time, at a specified cost, and at a specified level of service.

Week 3 & 4

Process Flow



Process Assessment and Improvement

- The existence of a software process is no guarantee that software will be delivered on time, or meet the customer's needs, or that it will exhibit long-term quality characteristics.
- Any **software process can be assessed to ensure that it meets a set of basic process criteria** that have been shown to be essential for successful software engineering.
Software processes and activities should be assessed using numeric measures or software analytics (metrics).

Prescriptive Process Model

- Prescriptive process models advocate an **orderly approach** to software engineering.
- That leads to two questions:
 - If prescriptive process models strive for structure and order, are they appropriate for a software world that thrives on change?
 - Yes, accomodate for change in the phase's structure gives order and prevents overlap
 - If we reject traditional process models and replace them with something less structured, do we make it impossible to achieve coordination and coherence in software work?
 - Yes, as there is no steps to follow as there is no structure. Each process requires atleast some structure and order.

Waterfall Model

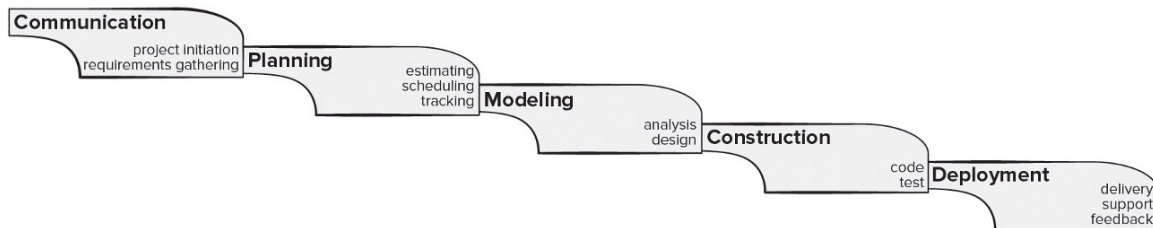
- A classic life cycle which suggests a **systemic, sequential approach** to software development
- The waterfall model, sometimes called the **linear sequential model**, suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment, culminating in ongoing support of the completed software.

When to use the Waterfall Model?

- Requirements are very well known
- Product definition is stable

- Technology is understood
- New version of an existing product
- Porting an existing product to a new platform.
- Smaller product

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Pros

- It is easy to understand and plan.
- It works for well-understood small projects.
- Analysis and testing are straightforward.

Cons

- It does not accommodate change well.
- Testing occurs late in the process.
- Customer approval is at the end.

Additional Notes:

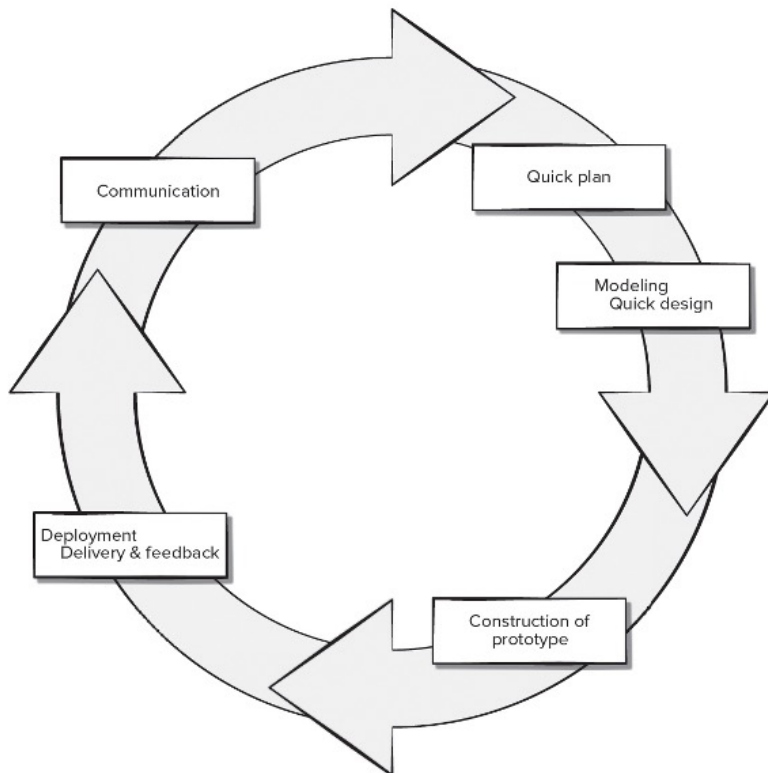
- Only time to meet client is at **communication**(Start) and **deployment**(End) stages.

Prototyping

- Often, a customer defines a set of general objectives for software but does not identify detailed requirements for functions and features. In other cases, the developer may be unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form that human-machine interaction should take. In these, and many other situations, a prototyping paradigm may offer the best approach.
- Although prototyping can be used as a stand-alone process model, it is more commonly **used as a technique that can be implemented within the**

context of any one of the process models noted in this chapter.

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Pros

- Reduced impact of requirement changes.
- Customer is involved early and often.
- Works well for small projects.
- Reduced likelihood of product rejection.

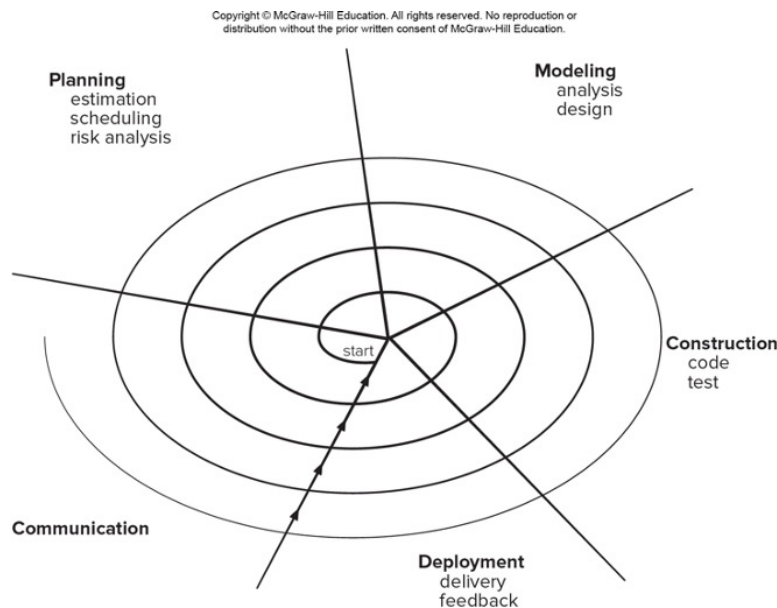
Cons

- Customer involvement may cause delays.
- Temptation to "ship" a prototype.
- Work lost in a throwaway prototype.
- Hard to plan and manage.

Evolutionary Process Model - Spiral Model

Characteristics

- Couples the **iterative nature of prototyping** with the **controlled and systematic aspects of the waterfall model**
- Provides the potential for rapid development of increasingly more complete versions of the software
- The spiral model is favored for large, expensive, and complicated projects.



Pros

- Continuous customer involvement.
- Development risks are managed.
- Suitable for large, complex projects.
- It works well for extensible products.

Cons

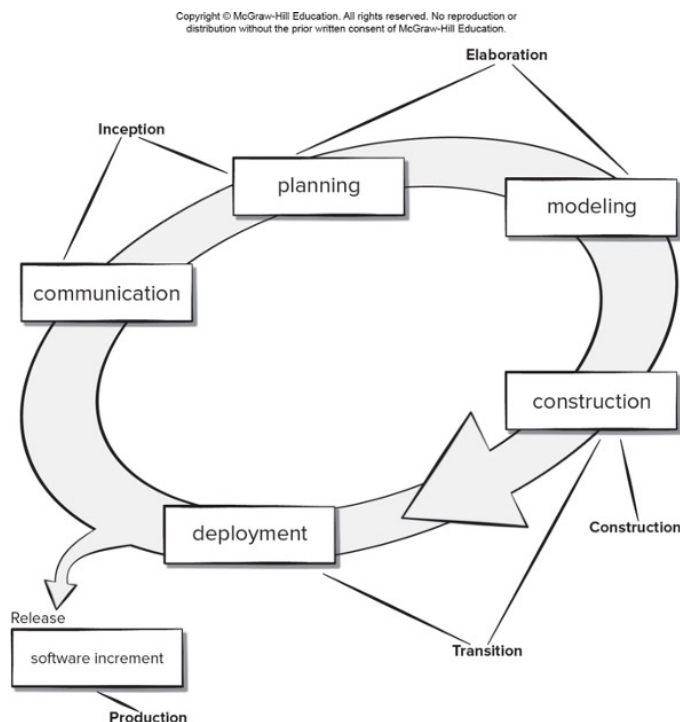
- Risk analysis failures can doom the project.
- Project may be hard to manage.
- Requires an expert development team.

Unified Process Model

- In some ways the Unified Process (UP) is an attempt to draw on the best features and characteristics of traditional software process models but

characterize them in a way that implements many of the best principles of agile software development.

- It suggests a process flow that is iterative and incremental, providing the evolutionary feel that is essential in modern software development.
- Unified Process—a “use-case driven, architecture- centric, iterative and incremental” software process closely aligned with the Unified Modeling Language (UML)



Pros

- Quality documentation emphasized.
- Continuous customer involvement.
- Accommodates requirements changes.
- Works well for maintenance projects.

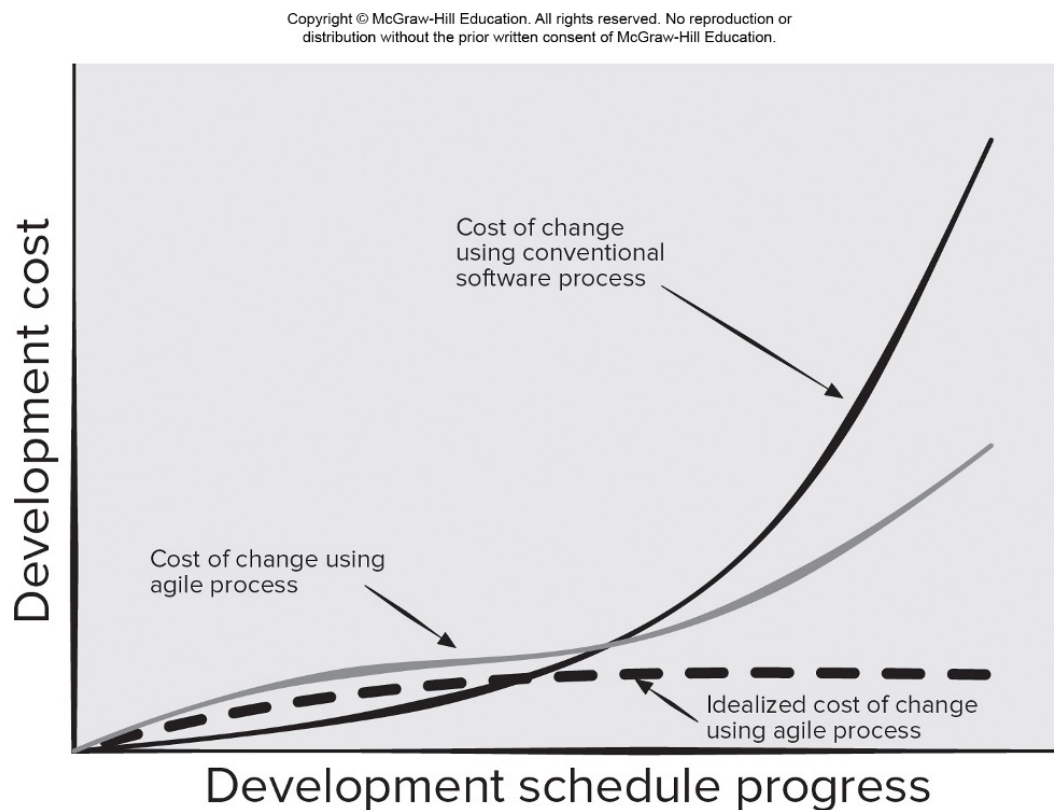
Cons

- Use cases are not always precise.
- Tricky software increment integration.
- Overlapping phases can cause problems.

What is Agility

- Effective (rapid and adaptive) response to change.
- Effective communicationA among all stakeholders.
- Drawing the customer onto the team.
- Organizing a team so that it is in control of the work performed.
- Rapid, incremental delivery of software.

Agility and Cost of Change



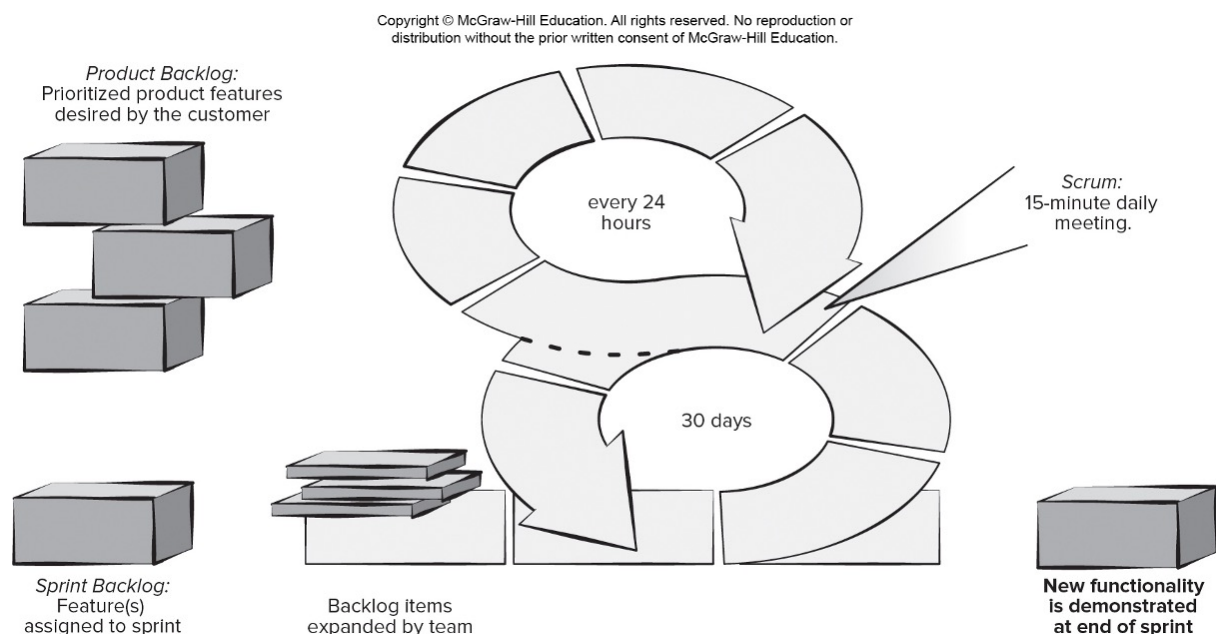
What is Agile Process

- Driven by customer descriptions of what is required (scenarios).
- Customer feedback is frequent and acted on.
- Recognizes that plans are short-lived.
- Develops software iteratively with a heavy emphasis on construction activities.
- Delivers multiple 'software increments' as executable prototypes.
- Adapts as project or technical changes occur.

Agility Principles

- Customer satisfaction is achieved by providing value through software that is delivered to the customer as rapidly as possible.
- Develop recognize that requirements will change and welcome changes.
- Deliver software increments frequently (weeks not months) to stakeholders to ensure feedback on their deliveries is meaningful.
- Agile team populated by motivated individuals using face-to-face communication to convey information.
- Team process encourages technical excellence, good design, simplicity, and avoids unnecessary work.
- Working software that meets customer needs is the primary goal.
- Pace and direction of the team's work must be "sustainable," enabling them to work effectively for long periods of time.
- An agile team is a "self-organizing team"—one that can be trusted develop well-structured architectures that lead to solid designs and customer satisfaction.
- Part of the team culture is to consider its work introspectively with the intent of improving how to become more effective its primary goal (customer satisfaction).

Scrum Framework



Details

- Backlog Refinement Meeting Developers work with stakeholders to create product backlog.
- Sprint Planning Meeting Backlog partitioned into “sprints” derived from backlog and next sprint defined.
- Daily Scrum Meeting Team members synchronize their activities and plan work day (15 minutes max).
- Sprint Review Prototype “demos” are delivered to the stakeholders for approval or rejection.
- Sprint Retrospective After sprint is complete, team considers what went well and what needs improvement.

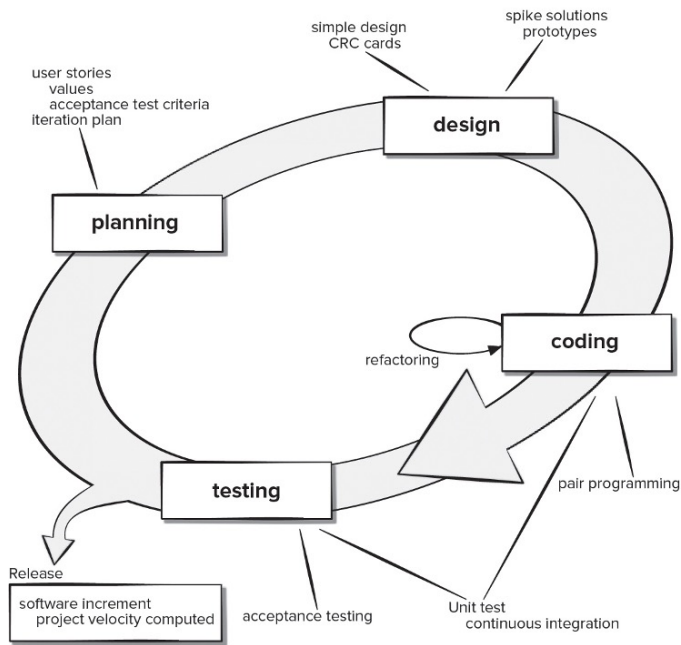
Pros

- Product owner sets priorities.
- Team owns decision making.
- Documentation is lightweight.
- Supports frequent updating.

Cons

- Difficult to control the cost of changes.
- May not be suitable for large teams.
- Requires expert team members.

Extreme Programming (XP) Model



XP Details

- **XP Planning** – Begins with user stories, team estimates cost, stories grouped into increments, commitment made on delivery date, computer project velocity.
- **XP Design** – Follows KIS principle, encourages use of CRC cards, design prototypes, and refactoring.
- **XP Coding** – construct unit tests before coding, uses pair.
- **XP Testing** – unit tests executed daily, acceptance tests define by customer.

Pros

- Emphasizes customer involvement.
- Establishes rational plans and schedules.
- High developer commitment to the project.
- Reduced likelihood of product rejection.

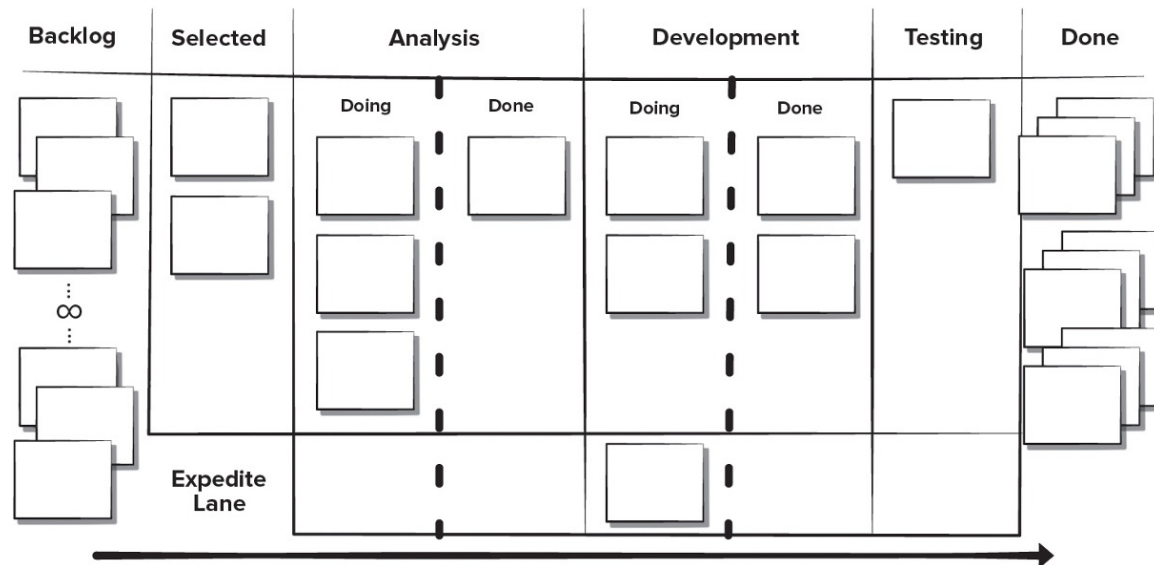
Cons

- Temptation to "ship" a prototype.
- Requires frequent meetings about increasing costs.

- Allows for excessive changes.
- Depends on highly skilled team members.

Kanban Framework

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Kanban Details

- Visualizing workflow using a Kanban board.
- Limiting the amount of work in progress at any given time.
- Managing workflow to reduce waste by understanding the current value flow.
- Making process policies explicit and the criteria used to define "done".
- Focusing on continuous improvement by creating feedback loops where changes are introduced.
- Make process changes collaboratively and involve all stakeholders as needed.

Pros

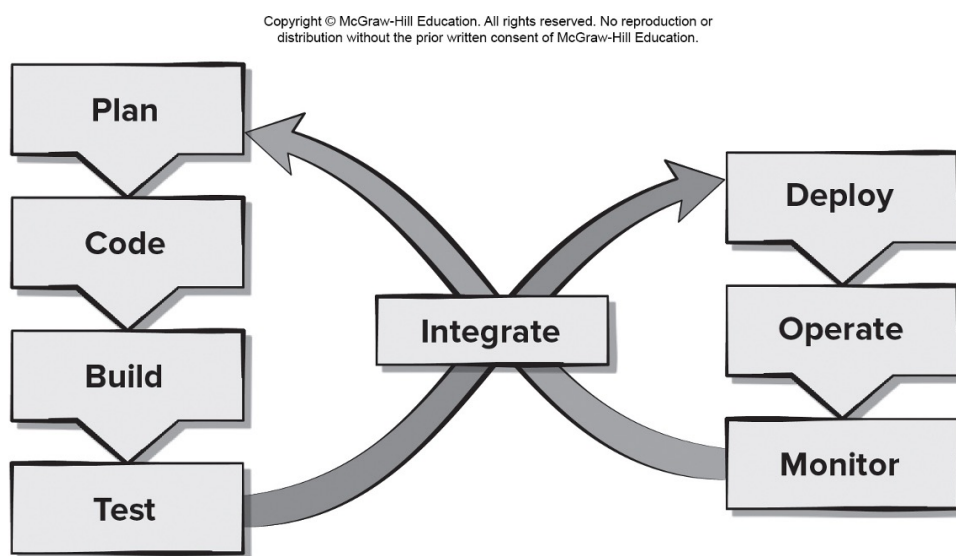
- Lower budget and time requirements.
- Allows early product delivery.
- Process policies written down.

- Continuous process improvement.

Cons

- Team collaboration skills determine success.
- Poor business analysis can doom the project.
- Flexibility can cause developers to lose focus.
- Developer reluctance to use measurement.

DevOps



DevOps Details

- **Continuous development.** Software delivered in multiple sprints.
- **Continuous testing.** Automated testing tools used prior to integration.
- **Continuous integration.** Code pieces with new functionality added to existing code running code.
- **Continuous deployment.** Integrated code is deployed to the production environment.
- **Continuous monitoring.** Team operations staff members proactively monitor software performance in the production environment.

Pros

- Reduced time to code deployment.

- Team has developers and operations staff.
- Team has end-to-end project ownership.
- Proactive monitoring of deployed product.

Cons

- Pressure to work on both old and new code.
- Heavy reliance on automated tools to be effective.
- Deployment may affect the production environment.
- Requires an expert development team.

Recommended Process Models

Adapting Process Models

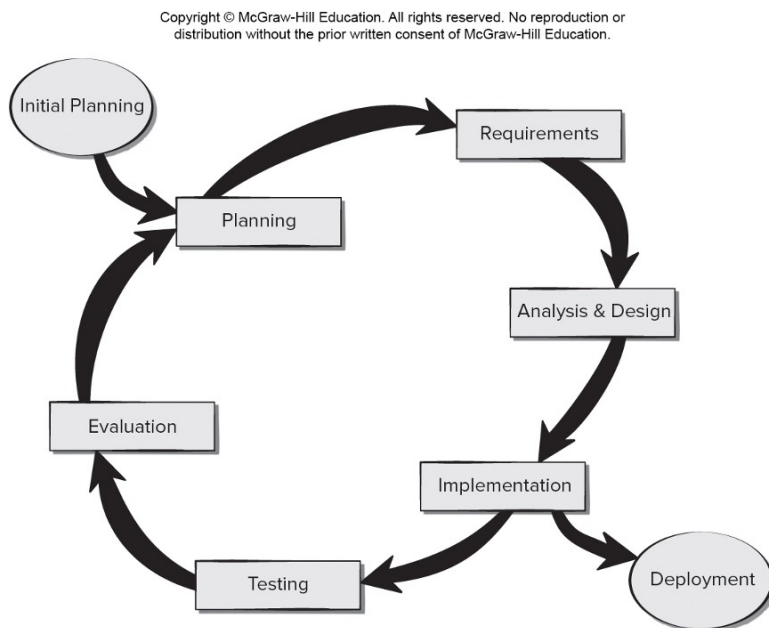
- Every software project needs a "road map" or "generic software process" of some kind.
- Every project is different, and every team is different.
- No single software engineering framework is appropriate for every software product.
- Any road map or generic process should be based on best industry practices.
- Developers and stakeholders adapt generic process models and tailor them to fit the current project, the skills of the team members, and the user needs.

Principles for Organizing Software Projects

1. It is risky to use a linear process model without ample feedback.
2. It is never possible nor desirable to plan big up-front requirements gathering.
3. Up-front requirements gathering may not reduce costs or prevent time slippage.
4. Appropriate project management is integral to software development.

5. Documents should evolve with the software and should not delay the start of construction.
6. Involve stakeholders early and frequently in the development process.
7. Testers need to become involved in the process prior to software construction.

Incremental Model for Prototype Design



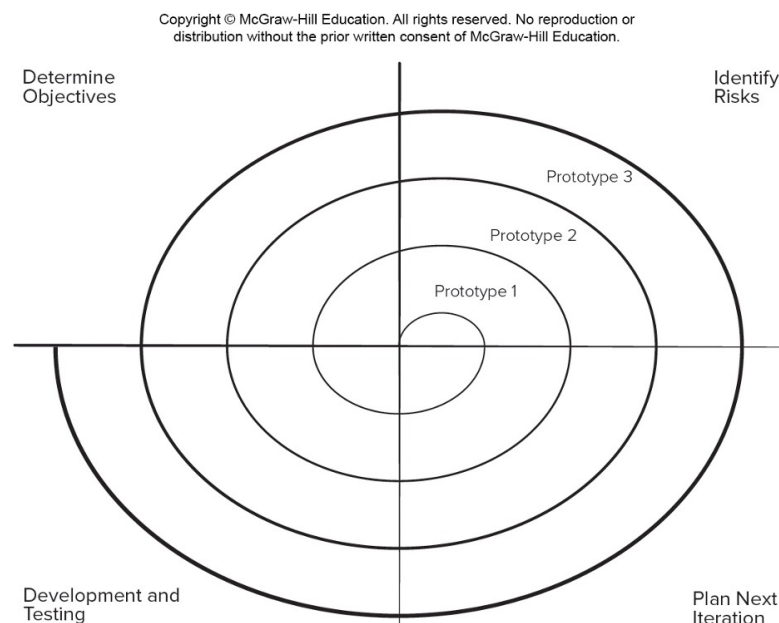
Characteristics of Agile Process Models

1. Not suitable for large high-risk or mission critical projects.
2. Minimal rules and minimal documentation.
3. Continuous involvement of testers.
4. Easy to accommodate product changes.
5. Depends heavily on stakeholder interaction.
6. Easy to manage.
7. Early delivery of partial solutions.
8. Informal risk management.
9. Built-in continuous process improvement.

Characteristics of Spiral Process Model

1. Not suitable for small, low-risk projects.
2. Several steps required, along with documentation done up front.
3. Early involvement of testers (might be done by outside team).
4. Hard to accommodate product changes until prototype completed.
5. Continuous stakeholder involvement in planning and risk assessment.
6. Requires formal project management and coordination.
7. Project end not always obvious.
8. Good risk management.
9. Process improvement handled at end of project.

Spiral Model for Prototype Design

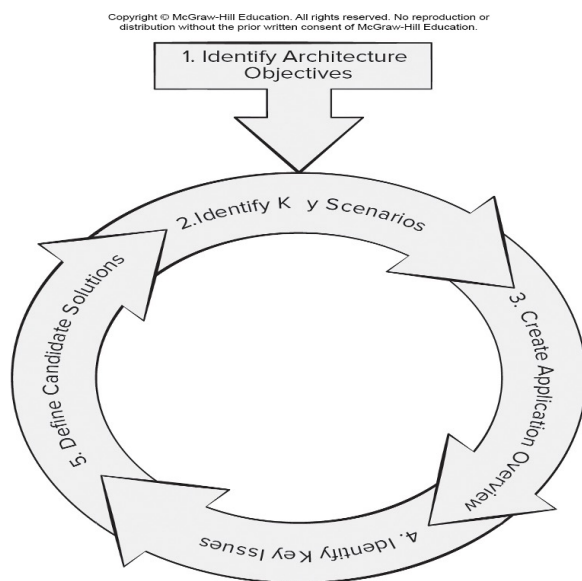


Agile Requirements Definition

1. Encourage active stakeholder participation by matching their availability and valuing their input.
2. Use simple models (for example, Post-it notes, fast sketches, user stories) to reduce barriers to participation.
3. Take time to explain your requirement representation techniques before using them.

4. Adopt stakeholder terminology and avoid technical jargon whenever possible.
5. Use a breadth-first approach to get the big picture of the project done before getting bogged down in details.
6. Developer and stakeholders refine requirements “just in time” as user stories are ready to be implemented.
7. Treat list of features like a prioritized list and implement the most important user stories first.
8. Collaborate closely with stakeholders and document requirements so they are useful to all when creating the next prototype.
9. Question the need to maintain models and documents not referred to in the future.
10. Ensure management support for stakeholder and resource availability during requirements definition.

Prototype Architectural Design



Elements of Agile Architectural Design

1. Focus on key quality attributes and incorporate them into prototypes as they are constructed.
2. Keep in mind that successful software products combine customer-visible features and the infrastructure needed to enable them.

3. Agile architectures enable code maintainability and evolvability if attention is paid to architectural decisions and quality issues.
4. Managing and synchronizing dependencies among functional and architectural requirements is needed to ensure evolving architecture will be ready for future increments.

Resource Estimation for Agile Spiral Model

1. Team should use historic data to develop an estimate of number of days needed to complete each of user stories known at the start of the project.
2. Loosely organize the user stories into sets that will make up each sprint planned to complete a prototype.
3. Sum the number of days to complete each sprint to provide an estimate for the duration of the total project.
4. Revise the estimate as requirements are added to the project or prototypes are delivered and accepted by the stakeholders.

First Prototype Guidelines

1. Transition from paper prototype to software design.
2. Prototype a user interface.
3. Create a virtual prototype.
4. Add input and output to your prototype.
5. Engineer your algorithms.
6. Test your prototype.
7. Prototype with deployment in mind.

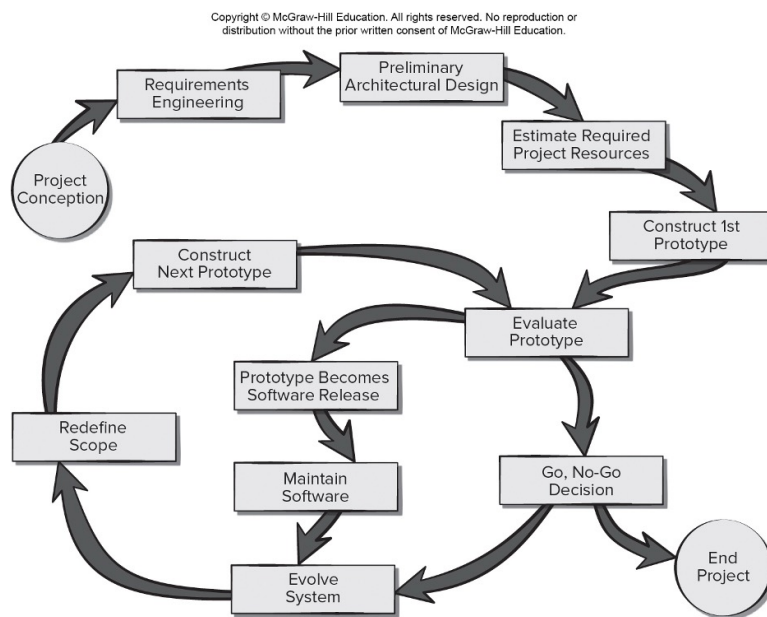
Prototype Evaluation

1. Provide scaffolding when asking for prototype feedback.
2. Test your prototype on the right people.
3. Ask the right questions.
4. Be neutral when presenting alternatives to users.
5. Adapt while testing.
6. Allow the user to contribute ideas.

Go No Go Decision

- A pass through the planning region follows the evaluation process.
- Revised cost estimates and schedule changes are proposed based on changes were requested when evaluating the current prototype.
- Risk of exceeding the budget and missing the project delivery date is assessed.
- Risk of failing to satisfy user expectations is also considered and discussed with the stakeholders and sometimes senior management.
- Goal of risk assessment is to get commitment from stakeholders and management to provide the resources needed to create the next prototype.

Recommended Prototype Evolutionary Process



Recommended Process Steps,

1. Requirements engineering.
 - Gather user stories from all stakeholders.
 - Have stakeholders describe acceptance criteria user stories.
2. Preliminary architectural design.
 - Make use of paper prototypes and models.
 - Assess alternatives using nonfunctional requirements.
 - Document architecture design decisions.

3. Estimate required project resources.
 - Use historic data to estimate time to complete each user story.
 - Organize the user stories into sprints.
 - Determine the number of sprints needed to complete the product.
 - Revise the time estimates as use stories are added or deleted.
4. Construct first prototype.
 - Select subset of user stories most important to stakeholders.
 - Create paper prototype as part of the design process.
 - Design a user interface prototype with inputs and outputs.
 - Engineer the algorithms needed for first prototypes.
 - Prototype with deployment in mind.
5. Evaluate prototype.
 - Create test cases while prototype is being designed.
 - Test prototype using appropriate users.
 - Capture stakeholder feedback for use in revision process.
6. Go, No-Go decision.
 - Determine the quality of the current prototype.
 - Revise time and cost estimates for completing development.
 - Determine the risk of failing to meet stakeholder expectations.
 - Get commitment to continue development.
7. Evolve system.
 - Define new prototype scope.
 - Construct new prototype.
 - Evaluate new prototype and include regression testing.
 - Assess risks associated with continuing evolution.
8. Release prototype.
 - Perform acceptance testing.
 - Document defects identified.
 - Share quality risks with management.
9. Maintain software.
 - Understand code before making changes.
 - Test software after making changes.
 - Document changes.
 - Communicate known defects and risks to all stakeholders.

Testing New Prototype

- Testing should be performed by developers using test cases created during the design process before programming was completed.
- Each user story has an acceptance criteria attached to it and it should guide the creation of the test cases to ensure the prototype meets customer needs.
- Prototypes need to be tested for defects and performance issues.
- Ensure that adding new features to evolutionary prototypes does not accidentally break features working correctly in the previous prototype (**regression testing**).

Release Candidates

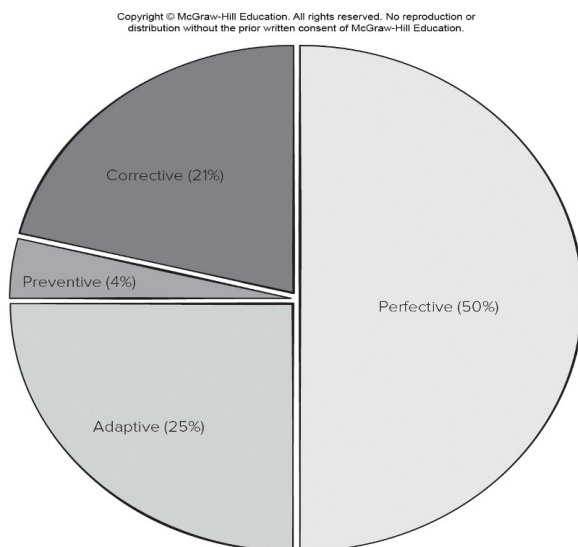
- A prototype considered as a release candidate is subjected to user acceptance testing in addition to testing conducted during prototype construction.
- User acceptance tests are based on acceptance criteria that were recorded as each user story was created and added to the product backlog.
- User feedback during acceptance testing should be organized by user-visible functions as portrayed via the user interface.
- Developers should make changes only if these changes will not delay the release of the prototype.
- If changes are made, they need to be verified in a second round of acceptance testing before moving on.
- The issues and lessons learned from creating the release candidate should be documented and considered by the developers and stakeholders as part of the project postmortem.
- This information should be considered before deciding to undertake future development of a software product.
- Lessons learned from the current product can help developers make better cost and time estimates for similar projects in the future.

Software Release Maintenance

- **Maintenance** - activities needed to keep software operational after it has been accepted and released in the end-user environment.

- **Corrective maintenance** - reactive modification of software to repair problems discovered after the software has been delivered.
- **Adaptive maintenance** - reactive modification of software after delivery to keep the software usable in a changing environment.
- **Perfective maintenance** - proactive modification of the software after delivery to provide new user features, better program code structure, or improved documentation.
- **Preventive maintenance** – proactive modification software after delivery to correct product faults before discovery by users.
- In agile process models much (but not all) of the maintenance work is preventive or perfective as new features are added.

Maintenance Effort Distribution



Week 5

Risk Management

Planning

These questions are addressed in the chapters that follow:

- How many activities are part of software planning?
- How can software metrics be used to manage a software project and the software process?

- How does a software team generate reliable estimates of effort, cost, and project duration?
- What techniques can be used to systematically assess the risks that can have an impact on project success?

Creating a Viable Software Plan

Software project management begins with a set of activities that are collectively called **project planning**. Before the project can begin, the software team estimates the work to be done, the resources that will be required, and the time that will elapse from start to finish.

Software project planning encompasses five major activities-

- Estimation,
- Scheduling,
- Risk analysis,
- Quality management planning, and
- Change management planning.

Project Planning Task Set

1. Establish project scope.
2. Determine feasibility.
3. Analyze risks (Chapter 26).
4. Define required resources.
 - Determine required human resources.
 - Define reusable software resources.
 - Identify environmental resources.
5. Estimate cost and effort.
 - Decompose the problem.
 - Develop two or more estimates.
 - Reconcile the estimates.
6. Develop an initial project schedule.
 - Establish a meaningful task set.

- Define a task network.
- Use scheduling tools to develop a time-line chart.
- Define schedule tracking mechanisms.

7. Repeat steps 1 to 6 to create a detailed schedule for each prototype as the scope of each prototype is defined.

What is Scope?

Software scope describes

- Functions and features to be delivered to end-users.
- Data input and output.
- Content presented to users of using the software.
- Performance, constraints, interfaces, and reliability that bound the system.

Scope is defined using one of two techniques:

- A narrative description of software scope is developed after communication with all stakeholders.
- A set of use-cases is developed by end-users.

Scope is the goals, limitations and constraints on the system. (Source: Project management)

Project Feasibility

Once scope has been identified (with the concurrence of the customer), it is reasonable to ask:

- Can we build software to meet this scope?
- Is the project feasible?
- You must try to determine if the system can be created using available technology, dollars, time, and other resources.
- Consideration of business need is important too - it does no good to build a high-tech system or product that no one wants.

Software Risks

- **Project risks** threaten the project plan.

- **Technical risks** threaten the quality and timeliness of the software to be produced.
- **Business risks** threaten the viability of the software to be built and often jeopardize the project or the product.
- **Known risks** are those that can be uncovered after careful evaluation project plan.
- **Predictable risks** are extrapolated from past project experience.
- **Unpredictable risks** can and do occur, but they are extremely difficult to identify in advance.

Risk Identification

- **Product size** - risks associated with the overall size of the software to be built or modified.
- **Business impact** - risks associated with constraints imposed by management or the marketplace.
- **Customer characteristics** - risks associated with the sophistication of the customer and the developer's ability to communicate with the customer in a timely manner.
- **Process definition** - risks associated with the degree to which the software process has been defined and is followed by the development organization.
- **Development environment** - risks associated with the availability and quality of the tools to be used to build the product.
- **Technology to be built** - risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system.
- **Staff size and experience** - risks associated with the overall technical and project experience of the software engineers who will do the work.

Assessing Project Risk

1. Have top software and customer managers formally committed to support the project?
2. Are end-users enthusiastically committed to the project and the system/product to be built?

3. Are requirements fully understood by the software engineering team and their customers?
4. Have customers been involved fully in the definition of requirements?
5. Do end-users have realistic expectations?
6. Is project scope stable?
7. Does the software engineering team have the right mix of skills?
8. Are project requirements stable?
9. Does the project team have experience with the technology to be implemented?
10. Is the number of people on the project team adequate to do the job?
11. Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built?

Risk Components

- **Performance risk** - the degree of uncertainty that the product will meet its requirements and be fit for its intended use.
- **Cost risk** - the degree of uncertainty that the project budget will be maintained.
- **Support risk** - the degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance.
- **Schedule risk** - the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time.

Risk Projection (risk estimation)

Risk projection attempts to rate each risk in two ways:

- Likelihood or probability that the risk is real.
- Consequences of the problems associated with the risk,

There are four risk projection steps:

1. Establish a scale that reflects the perceived likelihood of a risk.
2. Delineate the consequences of the risk.
3. Estimate the impact of the risk on the project and the product,

- Note the overall accuracy of the risk projection so that there will be no misunderstandings.

Risk Table

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Risk	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet exceptions	TR	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	

Impact values:
 1 – catastrophic
 2 – critical
 3 – marginal
 4 – negligible

Building Risk Table

- Estimate the probability of occurrence.
- Estimate the impact on the project on a scale of 1 to 5, where,
 1 = low impact on project success
 5 = catastrophic impact on project success
- Sort the table by probability and impact.

Risk Impact (Exposure)

- The overall risk exposure, RE, is determined using the following relationship [Ha198]:

$$RE = P \times C$$

where

- P is the probability of occurrence for a risk, and
- C is the cost to the project should the risk occur.

Risk Exposure Example

Risk identification. Only 70 percent of the software components scheduled for reuse will be used, the rest will have to be custom developed.

Risk probability. 80% (likely).

Risk impact. 60 reusable software components were planned. If only 70 percent can be used, 18 components would have to be developed from scratch. The average component is 100 L O C and the software engineering cost for each L O C is \$14.00, the overall cost (impact) to develop the components is $18 \times 100 \times 14 = \$25,200$.

Risk exposure. $RE = .80 \times 25,200 = \$20,200$

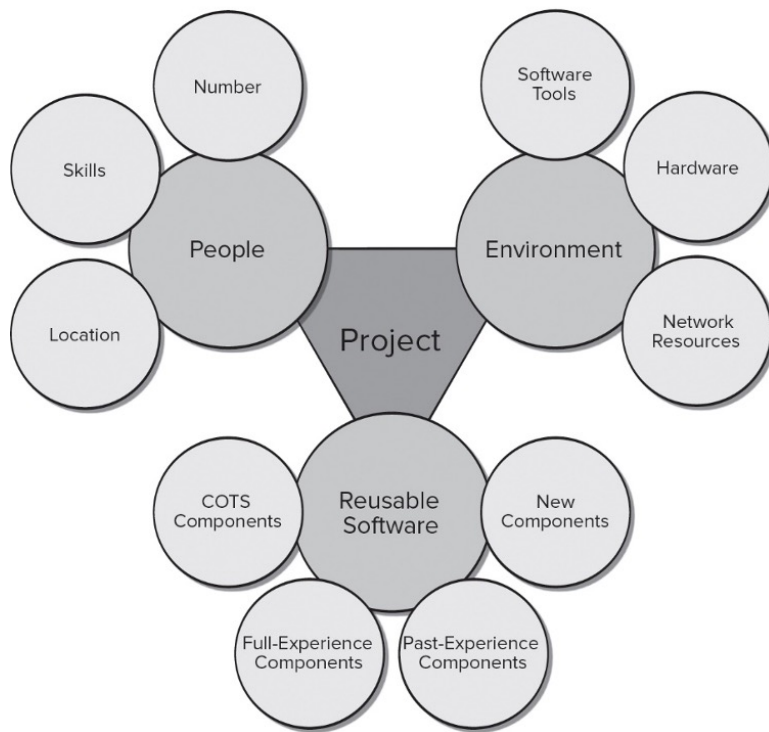
Risk Mitigation, Monitoring and Management

- **Mitigation** - how can we avoid the risk?
- **Monitoring** - what factors can we track that will enable us to determine if the risk is becoming more or less likely?
- **Management** - what contingency plans do we have if the risk becomes a reality?

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Risk information sheet			
Risk ID: P02-4-32	Date: 5 / 9 / 19	Prob: 80%	Impact: high
Description: Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.			
Refinement/context: Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards. Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components. Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.			
Mitigation/monitoring: 1. Contact third party to determine conformance with design standards. 2. Press for interface standards completion; consider component structure when deciding on interface protocol. 3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.			
Management/contingency plan/trigger: RE computed to be \$20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly. Trigger: Mitigation steps unproductive as of 7 / 1 / 19.			
Current status: 5 / 12 / 19: Mitigation steps initiated.			
Originator: D. Gagne		Assigned: B. Laster	

Project Resources



Reconciling Elements

- Any estimation technique must be checked by computing at least one other estimate using a different approach.
- If you have created multiple estimates they need to be compared and reconciled.
- If both estimates show agreement, there is good reason to believe that the estimates are reliable.
- Widely divergent estimates can often be traced to one of two causes:
 1. The scope of the project is not adequately understood or has been misinterpreted by the planner.
 2. Productivity data used for problem-based estimation techniques is inappropriate for the application or has been misapplied.

Problem-Based Estimation

- LOC and FP data are used in two ways during software project estimation:
 1. as estimation variables to "size" each element of the software
 2. as baseline metrics collected from past projects and used with other variable to develop cost and effort projections.

- When collecting productivity metrics for projects, be sure to establish a taxonomy of project types.
- Be sure that your estimates include the effort required to develop "infrastructure" software.
- Begin with a bounded statement of software scope.
- Decompose the statement of scope into problem functions that can each be estimated individually.
- L O C or F P is then estimated for each function.
- Baseline productivity metrics (For example, L O C/pm or F P/pm) are then applied to the appropriate estimation variable.
- Cost/effort for the function is derived using historic data.
- Function estimates are combined to produce an overall estimate for the entire project.

LOC-Based Estimation Table

Function	Estimated LOC
User-interface and control facilities (UICF)	2,300
Two-Dimensional geometric analysis (2DGA)	5,300
Three-Dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (GCDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
<i>Estimated lines of code</i>	33,200

- Average productivity for these systems is 620 LOC/pm
- Burdened labor rate is \$8000 per month.
- Cost per line of code is approximately \$13.
- Based on LOC estimates and historical data:
 1. estimated project cost is \$431,000
 2. estimated effort is 54 person-months

FP-Based Estimation Table

Information domain value	Opt.	Likely	Pess.	Est count.	Weight	F P count
Number of external inputs	20	24	30	24	4	96 (24 × 4 = 96)
Number of external outputs	12	14	22	14	5	70 (14 × 5 = 70)
Number of external enquiries	16	20	28	20	5	100 (20 × 5 = 100)
Number of internal logical files	4	4	5	4	10	40 (4 × 10 = 40)
Number of external interface files	2	2	3	2	7	14 (2 × 7 = 14)
Count Total						320

- To compute the FP equation:

$$FP_{estimated} = \text{count total} \times 0.65 + 0.01 \times \sum(F_i)$$
- For the purposes of this estimate, the complexity weighting factor is assumed to be average and the FP count total from the table is 320.
- Assume the sum of the 14 complexity factors $\sum(F_i)$ is 52.

$$[0.65 + 0.01 \times \sum(F_i)] = 1.17$$
- The estimated number of FP can be computed:

$$FP_{estimated} = \text{count total} \times [0.65 + 0.01 \times \sum(F_i)] = 375$$
- If the historic cost per FP is approximately \$1,230 then total estimated project cost is \$461,000 estimated effort is 58 person-months.

Process-Based Estimation Table

Activity →	CC	Planning	Risk analysis	Engineering		Construction release		CE	Totals
Task →				Analysis	Design	Code	Test		
Function									
↓									
UICF				0.50	2.50	0.40	5.00	n/a	8.40
2DGA				0.75	4.00	0.60	2.00	n/a	7.35
3DGA				0.50	4.00	1.00	3.00	n/a	8.50
CGDF				0.50	3.00	1.00	1.50	n/a	6.00
DBM				0.50	3.00	0.75	1.50	n/a	5.75
PCF				0.25	2.00	0.50	1.50	n/a	4.25
DAM				0.50	2.00	0.50	2.00	n/a	5.00
Totals	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
% effort	1%	1%	1%	8%	45%	10%	36%		

- Process-based estimation begins with a delineation of software functions obtained from the project scope.
- A series of framework activities are performed for each function.
- Functions and related framework activities may be represented as part of a table with tasks as columns and rows as functions.
- The effort estimates (for example, person-months) are entered as the matrix cell values.
- Average labor rates (that is, cost/unit effort) are then applied to the effort estimated for each process activity.
- Based on an average burdened labor rate of \$8,000 per month, the total estimated project cost is \$368,000 and the estimated effort is 46 person-months based on the matrix entries.

Use Case Point Estimation

Computation of use case points takes the following into account:

- The number and complexity of the use cases in the system.
- The number and complexity of the actors on the system.
- Various nonfunctional requirements not written as use cases.

- The environment in which the project will be developed.

1. $UCP = (UUCW + UAW) * TCF * ECF$
2. UUCW - unadjusted sum of use case weights
3. UAW - unadjusted sum of actor weight
4. TCF - technical complexity 13 factors
5. ECF - environment complexity 8 factors

Example

The engineering subsystem group is described by 14 average use cases and 8 simple use cases. And the infrastructure subsystem is described with 10 simple use cases.

$$UUCW = (16 \text{ use cases} \times 15) + [(14 \text{ use cases} \times 10) + (8 \text{ use cases} \times 5)] + (10 \text{ use cases} \times 5) = 470$$

There are 8 simple actors, 12 average actors, and 4 complex actors.

$$UAW = (8 \text{ actors} \times 1) + (12 \text{ actors} \times 2) + (4 \text{ actors} \times 3) = 44$$

After evaluation of the technology and the environment,

$$TCF = 1.04$$

$$ECF = 0.96$$

$$UCP = (470 + 44) \times 1.04 \times 0.96 = 513$$

The development group produces **85 LOC per UCP**.

An estimate of the overall size of the CAD project is

43,600 LOC.

Using

620 LOC/pm as the average productivity for systems of this type and a burdened labor rate of \$8,000 per month, and the cost per line of code is approximately **\$13**.

Based on the use case estimate and the historical productivity data:

Total estimated project cost is \$552,000. **$85 \times 513 \times 13$**

Estimated effort is about 70 person-months.

$$\mathbf{85 \times 513 / 620}$$

Agile Project Estimation

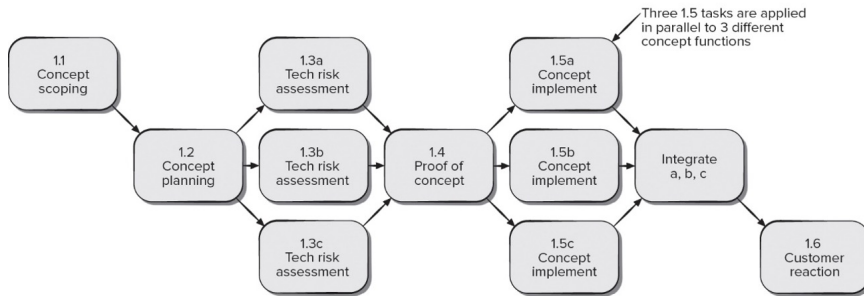
1. Each user story is considered separately for estimation purposes.

2. Each user story is decomposed into the set of software engineering tasks that will be required to develop it.
- 3a. Each task is estimated separately (historic data, empirical model, experience, or planning poker).
- 3b. Alternatively, the "volume" of the user story can be estimated in LOC, FP, or use case count.
- 4a. Estimates for each task are summed to estimate the user story.
- 4b. Alternatively, the volume translated into effort using historical data.
5. Effort estimates for all user stories are summed to create effort estimate for the increment.

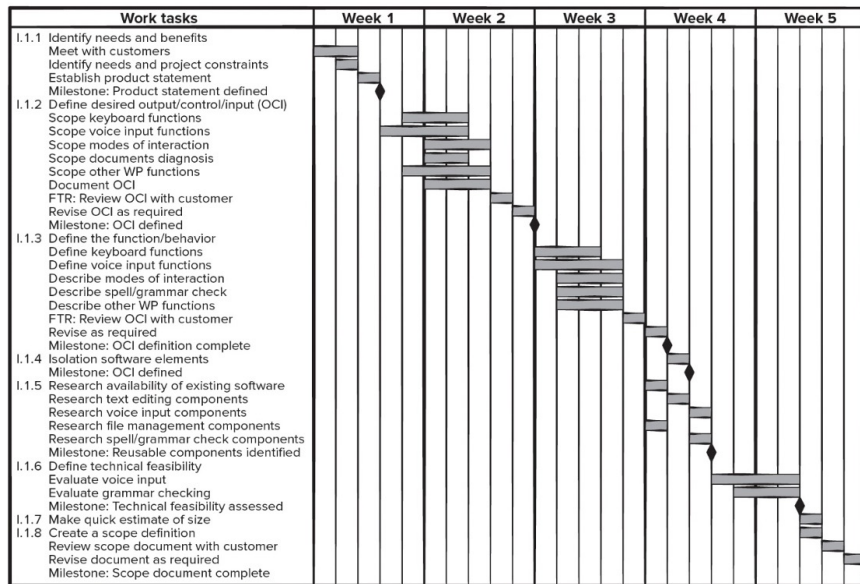
Scheduling Principles

- **Compartmentalization.** The project must be compartmentalized by decomposing the product and the process.
- **Interdependency.** The interdependency of each compartmentalized activity or task must be determined.
- **Time allocation.** Each task must be allocated some number of work units and assigned a start date and a completion date.
- **Effort validation.** Ensure that no more than the allocated number of people has been scheduled at any given time.
- **Defined responsibilities.** Every task that is scheduled should be assigned to a specific team member.
- **Defined outcomes.** Every task should have a defined outcome.
- **Defined milestones.** Every task should be associated with a project milestone.

Task Network (Activity Network)



Timeline Chart (Gantt Chart)



Project Table for Project Tracking

Work tasks	Planned start	Actual start	Planned complete	Actual complete	Assigned person	Effort allocated	Notes
I.1.1 Identify needs and benefits							
Meet with customers	wk1, d1	wk1, d1	wk1, d2	wk1, d2	BLS	2 p-d	Scoping will require more effort/time
Identify needs and project constraints	wk1, d2	wk1, d2	wk1, d2	wk1, d2	JPP	1 p-d	
Establish product statement	wk1, d3	wk1, d3	wk1, d3	wk1, d3	BLS/	1 p-d	
Milestone: Product statement defined	wk1, d3	wk1, d3	wk1, d3	wk1, d3			
I.1.2 Define desired output/control/input (OCI)							
Scope keyboard functions	wk1, d4	wk1, d4	wk2, d2		BLS	1.5 p-d	
Scope voice input functions	wk1, d3	wk1, d3	wk2, d2		JPP	2 p-d	
Scope modes of interaction	wk2, d1		wk2, d3		MLL	1 p-d	
Scope documents diagnosis	wk2, d1		wk2, d2		BLS	1.5 p-d	
Scope other WP functions	wk1, d4	wk1, d4	wk2, d3		JPP	2 p-d	
Document OCI	wk2, d1		wk2, d3		MLL	3 p-d	
FTR: Review OCI with customer	wk2, d3		wk2, d3		all	3 p-d	
Revise OCI as required	wk2, d4		wk2, d4		all	3 p-d	
Milestone: OCI defined	wk2, d5		wk2, d5				
I.1.3 Define the Function/behavior							

Schedule Tracking

- Conducting periodic project status meetings in which each team member reports progress and problems.

- Evaluating the results of all reviews conducted throughout the software engineering process.
- Determining whether formal project milestones have been accomplished by the scheduled date.
- Comparing the actual start date to the planned start date for each project task listed in the project resource table.
- Meeting informally with practitioners to obtain their subjective assessment of progress to date and problems on the horizon.
- Tracking the project velocity, which is a way of seeing how quickly the development team is clearing the user story backlog.

Week 6

Introduction to UML

System Modelling

- System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modeling has now come to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).
- System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers.

UML

- The Unified Modeling Language (UML) is "a standard language for writing software blueprints. UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system". In other words, just as building architects create blueprints to be used by a construction company, software architects create UML diagrams to help software developers build the software.
- If you understand the vocabulary of UML (the diagrams' pictorial elements and their meanings), you can much more easily understand and specify a system and explain the design of that system to others.

UML Diagram Type

- **Activity diagrams**, which show the activities involved in a process or in data processing .
- **Use case diagrams**, which show the interactions between a system and its environment.
- **Sequence diagrams**, which show interactions between actors and the system and between system components.
- **Class diagrams**, which show the object classes in the system and the associations between these classes.
- **State diagrams**, which show how the system reacts to internal and external events.

Use of graphical models

- As a means of facilitating discussion about an existing or proposed system
 - Incomplete and incorrect models are OK as their role is to support discussion.
- As a way of documenting an existing system
 - Models should be an accurate representation of the system but need not be complete.
- As a detailed system description that can be used to generate a system implementation
 - Models have to be both correct and complete.

Interaction Models

- Modeling user interaction is important as it helps to identify user requirements.
- Modeling system-to-system interaction highlights the communication problems that may arise.
- Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.
- Use case diagrams and sequence diagrams may be used for interaction modeling.

Use Case Modeling

- Use cases were developed originally to support requirements elicitation and now incorporated into the UML.
- Each use case represents a discrete task that involves external interaction with a system.
- Actors in a use case may be people or other systems.
- Represented diagrammatically to provide an overview of the use case and in a more detailed textual form.

Transfer-data use case

- A use case in the Mentcare system



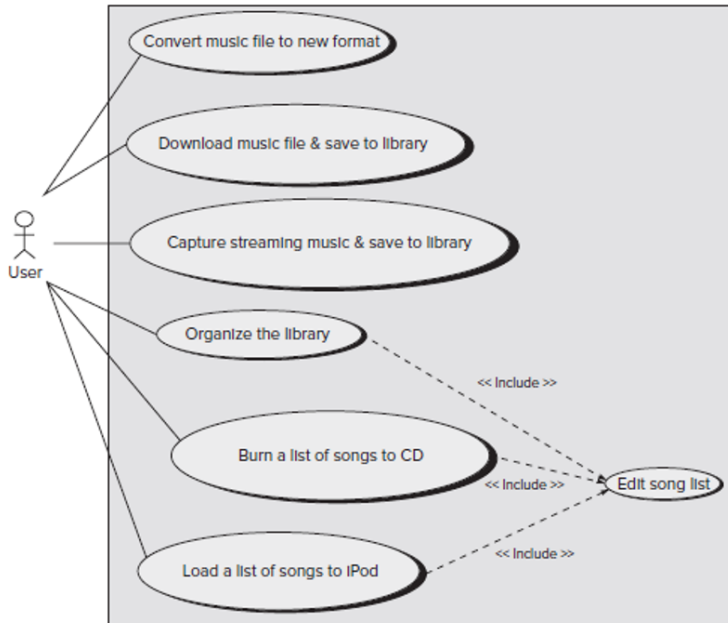
Tabular description of the 'Transfer data' use-case

MHC-PMS: Transfer data	
Actors	Medical receptionist, patient records system (PRS)
Description	A receptionist may transfer data from the <u>Mentcase</u> system to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Use-Case Diagram Example

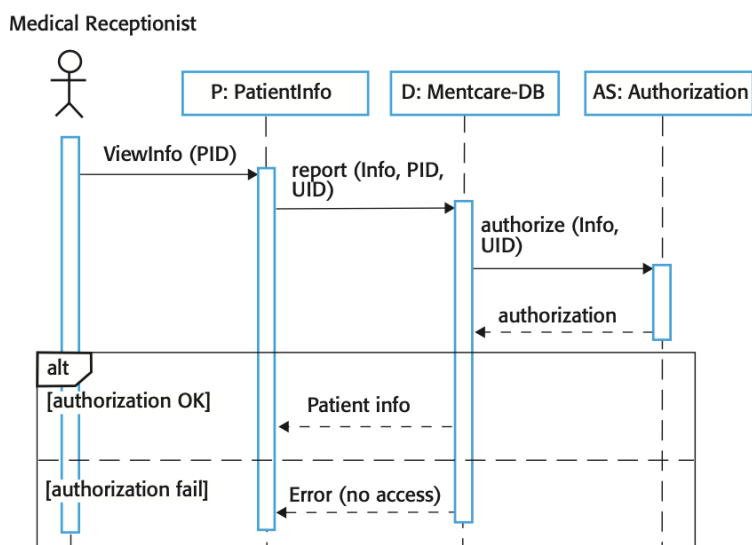
- To give you a feeling for how use cases and use-case diagrams work, we'll create some for a software application for managing an online digital music store. Some of the things the software might do include:
 - Download an MP3 music file and store it in the application's library.
 - Capture streaming music and store it in the application's library.

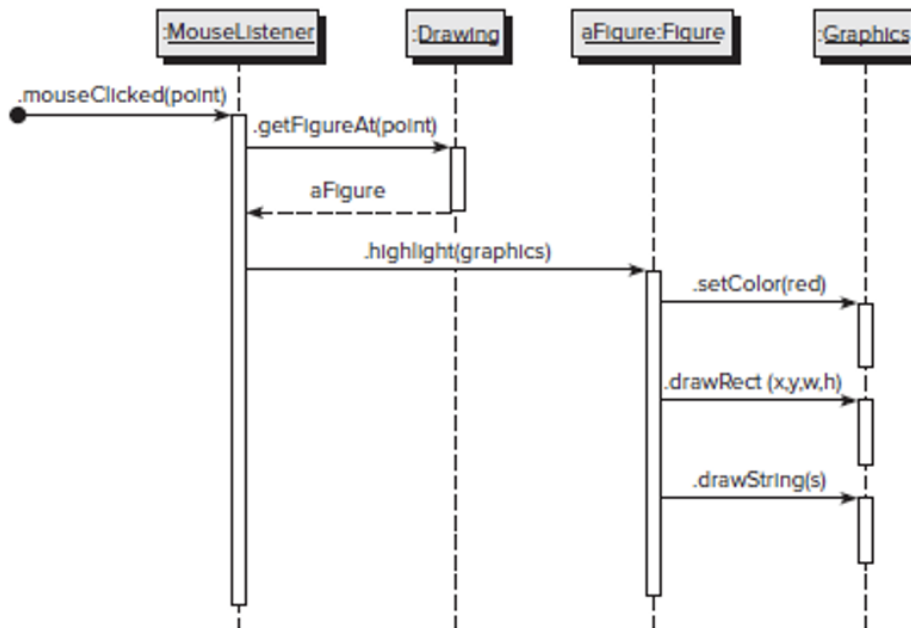
- Manage the application's library (e.g., delete songs or organize them in playlists).
- Burn a list of the songs in the library onto a CD.
- Load a list of the songs in the library onto an iPod or MP3 player.
- Convert a song from MP3 format to AAC format and vice versa.
- A use case describes how a user interacts with the system by defining the steps required to accomplish a specific goal (e.g., burning a list of songs onto a CD). Variations in the sequence of steps describe various scenarios (e.g., what if all the songs in the list don't fit on one CD?).
- A UML use-case diagram is an overview of all the use cases and how they are related. It provides a big picture of the functionality of the system.
- In this diagram, the stick figure represents an **actor** that is associated with one category of user (or other interaction element).
- In the use-case diagram, the use cases are displayed as **ovals**. The actors are connected by lines (**relationship**) to the use cases that they carry out.
- Also that the use cases are placed in a rectangle (System) but the actors are not. This rectangle is a visual reminder of the **system boundaries** and that the actors are outside the system.
- Some use cases in a system might be related to each other. To avoid duplication in use cases, it is usually better to create a new use case representing the duplicated activity and then let the other use cases include this new use case as one of their steps.
- Such inclusion is indicated in use-case diagrams, by means of a dashed arrow labeled «**include**» connecting a use case with an included use case.
- Another useful connection is called **extend**. It is optional use-case.



Sequence Diagram

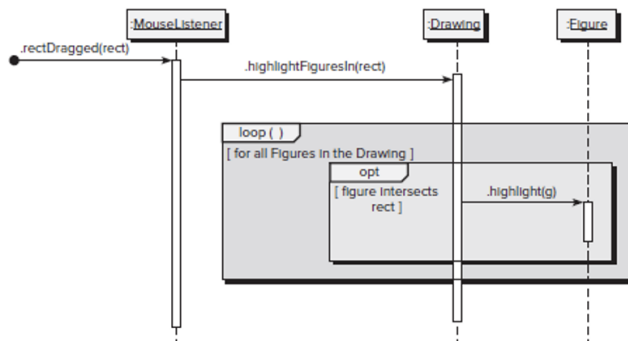
- Sequence diagrams are part of the UML and are used to model the interactions between the actors and the objects within a system.
- A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.
- The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.
- Interactions between objects are indicated by annotated arrows.
- System sequence diagram is the first diagram to show the system where whole system is represented by a box or rectangle.





- In Figure above, you see a sequence diagram for a drawing program. The diagram shows the steps involved in highlighting a figure in the drawing when it has been clicked.
- Each box in the row at the top of the diagram usually corresponds to an object, although it is possible to have the boxes model other things, such as classes. If the box represents an object (as is the case in all our examples), then inside the box you can optionally state the type of the object preceded by the colon. You can also precede the colon and type by a name for the object, as shown in the third box.
- Below each box there is a dashed line called the lifeline of the object. The vertical axis in the sequence diagram corresponds to time, with time increasing as you move downward.
- A sequence diagram shows method calls using horizontal arrows from the caller to the callee, labeled with the method name and optionally including its parameters, their types, and the return type.
- When an object is executing a method (that is, when it has an activation frame on the stack), you can optionally display a white bar, called an **activation bar**, down the object's lifeline.
- The diagram can also optionally show the return from a method call with a dashed arrow and an optional label.

- A common practice, as shown, is to leave off the return arrow when a void method has been called, since it clutters up the diagram while providing little information of importance.

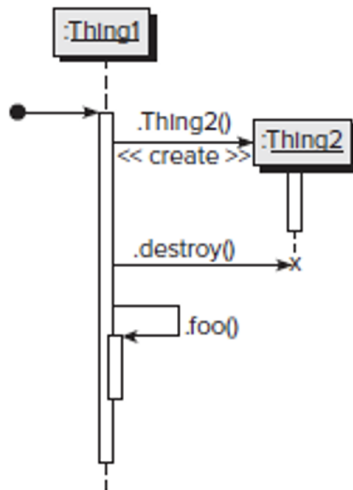


- The diagram of simple sequence diagram is very straightforward and contains no conditionals or loops. If logical control structures are required, it is probably best to draw a separate sequence diagram for each case. That is, if the message flow can take two different paths depending on a condition, then draw two separate sequence diagrams, one for each possibility.
- If you insist on including loops, conditionals, and other control structures in a sequence diagram, you can use interaction frames, which are rectangles that surround parts of the diagram and that are labeled with the type of control structures they represent. Figure above illustrates this, showing the process involved in highlighting all figures inside a given rectangle.

There are many other special features that can be included in a sequence diagram. For example:

1. You can distinguish between synchronous and asynchronous messages. Synchronous messages are shown with solid arrowheads, while asynchronous messages are shown with stick arrowheads.
2. You can show an object sending itself a message with an arrow going out from the object, turning downward, and then pointing back.
3. You can show object creation by drawing an arrow appropriately labeled (for example, with a «create» label) to an object's box. In this case, the box will appear lower in the diagram than the boxes corresponding to objects already in existence when the action begins.

4. You can show object destruction by a big X at the end of the object's lifeline

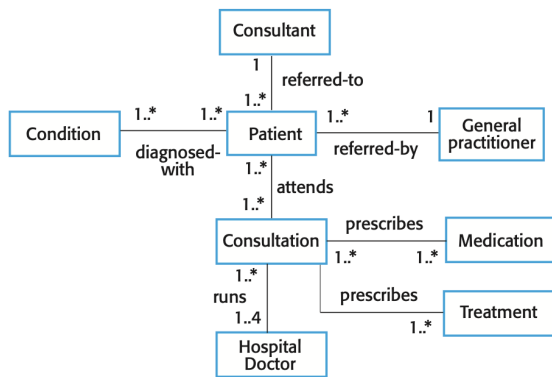


Structural Models

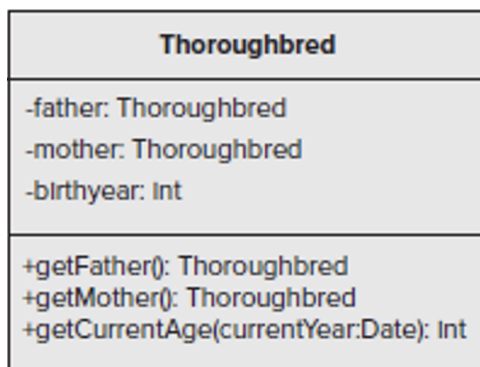
- Structural models of software display the organization of a system in terms of the components that make up that system and their relationships.
- Structural models may be static models, which show the structure of the system design, or dynamic models, which show the organization of the system when it is executing.
- You create structural models of a system when you are discussing and designing the system architecture.

Class Diagrams

- Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes.
- An object class can be thought of as a general definition of one kind of system object.
- An association is a link between classes that indicates that there is some relationship between these classes.
- When you are developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, doctor, etc.



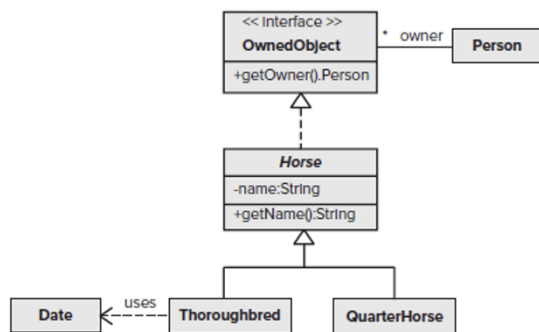
- The main elements of a class diagram are boxes, which are the icons used to represent classes and interfaces. Each box is divided into horizontal parts.
- The top part contains the **name of the class**. The middle section lists the **attributes** of the class. Attributes can be values that the class can compute from its instance variables or values that the class can get from other objects of which it is composed.
- The third section of the class diagram contains the **operations** or **behaviors** of the class. An operation refers to what objects of the class can do. It is usually implemented as a **method** of the class.



- Figure presents a simple example of a Thoroughbred class that models thoroughbred horses. It has three attributes displayed—mother, father, and birth year. The diagram also shows three operations: getCurrentAge(), getFather(), and getMother(). There may be other suppressed attributes and operations not shown in the diagram.
- Each attribute can have a name, a type, and a level of visibility. The type and visibility are optional. The type follows the name and is separated from

the name by a colon. The visibility is indicated by a preceding -, #, ~, or +, indicating, respectively, private, protected, package, or public visibility.

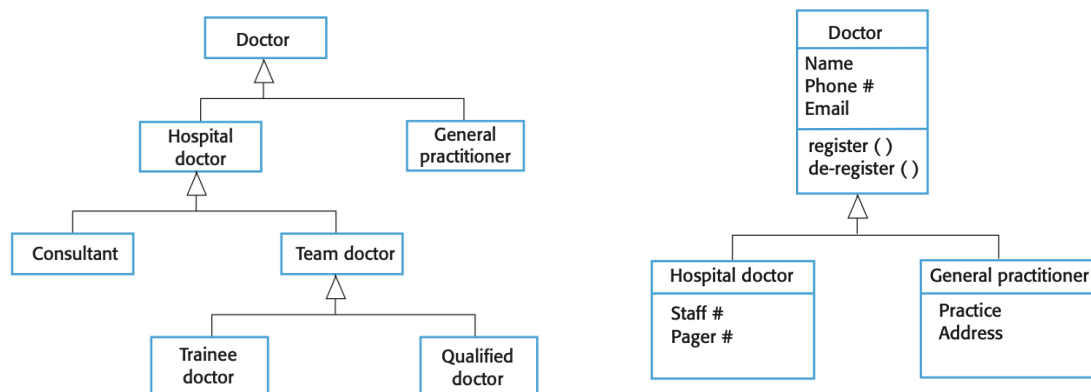
- An abstract class or abstract method is indicated by the use of italics for the name in the class diagram. See the Horse class, for an example. An interface is indicated by adding the phrase "«interface»" (called a stereotype) above the name.



- An **association** between two classes means that there is a structural relationship between them. Associations are represented by solid lines. An association has many optional parts. It can be labeled, as can each of its ends, to indicate the role of each class in the association.
- For example, there is an association between **OwnedObject** and **Person** in which the **Person** plays the role of owner.
- Arrows on either or both ends of an association line indicate **navigability**.
- Also, each end of the association line can have a multiplicity value displayed.
- An association with an arrow at one end indicates one-way navigability. The arrow means that from one class you can easily access the second associated class to which the association points, but from the second class, you cannot necessarily easily access the first class.
- An association with no arrows usually indicates a two-way association.
- The multiplicity of one end of an association means the number of objects of that class associated with the other class. A multiplicity is specified by a nonnegative integer or by a range of integers. A multiplicity specified by "0..1" means that there are 0 or 1 objects on that end of the association. What about '*' placed at the end?

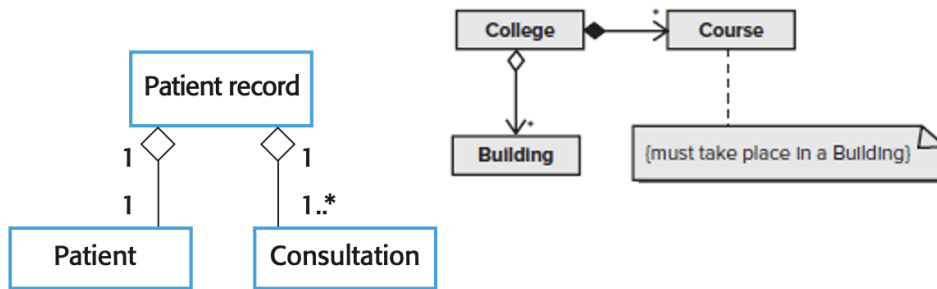
Generalisation

- Generalization is an everyday technique that we use to manage complexity.
- Rather than learn the detailed characteristics of every entity that we experience, we place these entities in more general classes (animals, cars, houses, etc.) and learn the characteristics of these classes.
- This allows us to infer that different members of these classes have some common characteristics e.g. squirrels and rats are rodents.



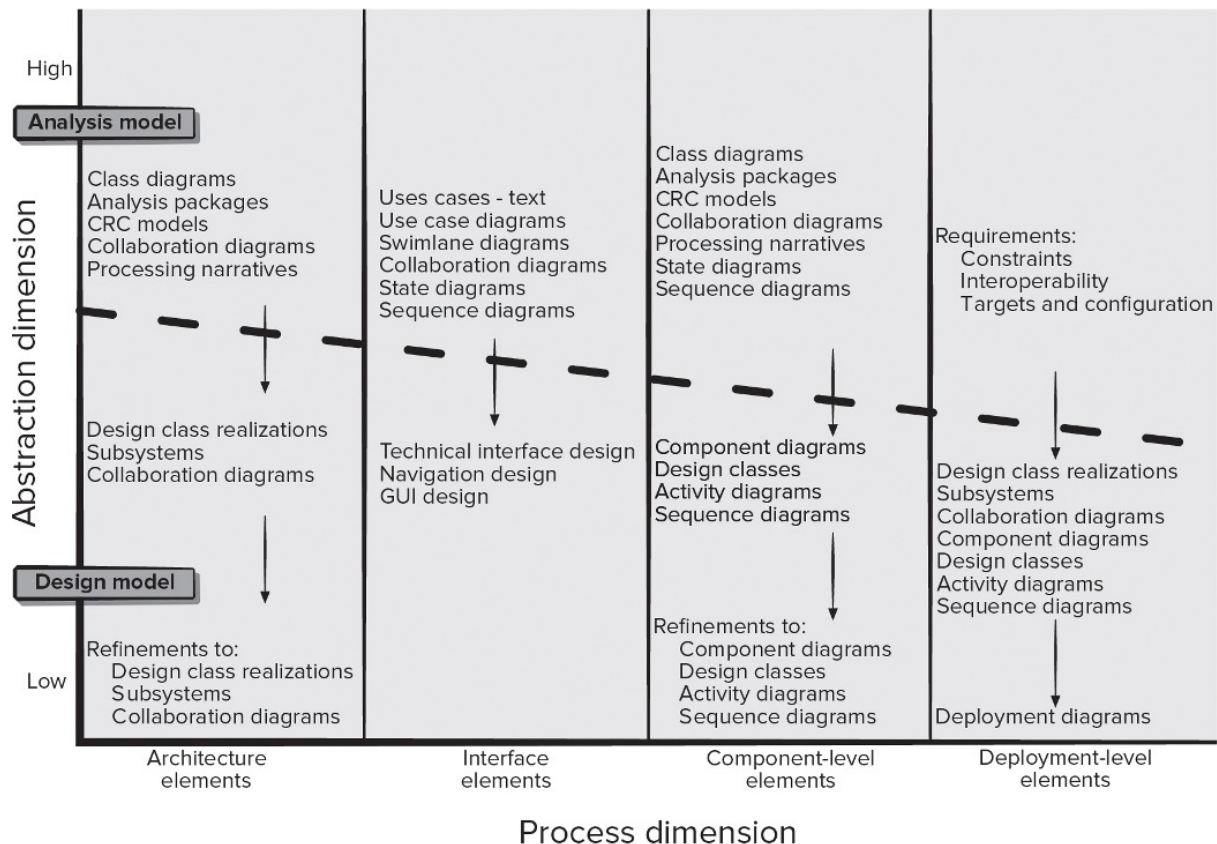
Object Class

- An **aggregation** model shows how classes that are collections are composed of other classes. Aggregation models are similar to the part-of relationship in semantic data models.
- An aggregation is a special kind of association indicated by a hollow diamond on one end of the icon. It indicates a "whole/part" relationship, in that the class to which the arrow points is considered a "part" of the class at the diamond end of the association.
- A **composition** is an aggregation indicating strong ownership of the parts. In a composition, the parts live and die with the owner because they have no role in the software system independent of the owner.



Analysis and Design Model

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.



Week 7

Understanding Requirements

Requirements Engineering

1. Inception - establish a basic understanding of the problem, the people who want a solution, and the nature of the solution that is desired, important to establish effective customer and developer communication.

2. Elicitation - elicit requirements and business goals from all stakeholders.
3. Elaboration - focuses on developing a refined requirements model that identifies aspects of software function, behavior, and information.
4. Negotiation—agree on the scope of a deliverable system that is realistic for developers and customers.
5. Specification—can be any or all of the following: written documents, graphical models, mathematical models, usage scenarios, prototypes.
6. Validation—Requirements engineering work products produced during requirements engineering are assessed for quality and consistency.
7. Requirements management – set of traceability activities to help the project team identify, control, and track requirements and their changes to requirements as the project proceeds.

Non-Functional Requirements

Non-Functional Requirement (N F R) – quality attribute, performance attribute, security attribute, or general system constraint.

A two-phase process is used to determine which N F R's are compatible:

- The first phase is to create a matrix using each N F R as a column heading and the system S E guidelines a row labels.
- The second phase is for the team to prioritize each N F R using a set of decision rules to decide which to implement by classifying each N F R and guideline pair as complementary, overlapping, conflicting, or independent.

Establishing the Groundwork

Identify stakeholders.

- "who else do you think I should talk to?"

Recognize multiple points of view.

Work toward collaboration.

The first questions.

- Who is behind the request for this work?
- Who will use the solution?
- What will be the economic benefit of a successful solution?

- Is there another source for the solution that you need?

Collaborative Requirements Gathering

- Meetings (real or virtual) are conducted and attended by both software engineers and other stakeholders.
- Rules for preparation and participation are established.
- Agenda is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas.
- A "facilitator" (customer, developer, or outsider) controls the meeting.
- A "definition mechanism" (worksheets, flip charts, wall stickers or virtual forum) is used.
- Goal is to identify the problem, propose solution elements, and negotiate different approaches.

Elicitation Work Products

- Statement of need and feasibility.
- Bounded statement of scope for the system or product.
- List of customers, users, and other stakeholders who participated in requirements elicitation,
- Description of the system's technical environment.
- List of requirements (preferably organized by function) and the domain constraints that apply to each.
- Set of usage scenarios (written in stakeholders' own words) that provide insight into the use of the system or product under different operating conditions.

Use Case Definition

A collection of user scenarios that describe the thread of usage of a system
Each scenario is described from the point-of-view of an "actor" - a person or device that interacts with the software in some way
Each scenario answers the following questions:

- Who is the primary actor, the secondary actor (s)?
- What are the actor's goals?

- What preconditions should exist before the story begins?
- What main tasks or functions are performed by the actor?
- What extensions might be considered as the story is described?
- What variations in the actor's interaction are possible?
- What system information will the actor acquire, produce, or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

Analysis Model Elements

- **Analysis model** provides a description of the required informational, functional, and behavioral domains for a computer-based system.
- **Scenario-based elements** – functional descriptions are express in the customers own words and user stories and as interactions of actors with the system expressed using U M L use case diagrams.
- **Class-based elements** – collections of attributes and behaviors implied by the user stories and expressed using U M L class diagrams (information domain).
- **Behavioral elements** – may be expressed using U M L state diagrams as inputs causing state changes.

Analysis Patterns

Pattern name: A descriptor that captures the essence of the pattern.

Intent: Describes what the pattern accomplishes or represents.

Motivation: A scenario that illustrates how the pattern can be used to address the problem.

Forces and context: A description of external issues (forces) that can affect how the pattern is used and the external issues that will be resolved when the pattern is applied.

Solution: A description of how the pattern is applied to solve the problem with an emphasis on structural and behavioral issues.

Consequences: Addresses what happens when the pattern is applied and what trade-offs exist during its application.

Design: Discusses how the analysis pattern can be achieved through the use of known design patterns.

Known uses: Examples of uses within actual systems.

Related patterns: One or more analysis patterns that are related to the named pattern because:

1. it is commonly used with the named pattern;
2. it is structurally similar to the named pattern;
3. it is a variation of the named pattern.

Negotiating Requirements

Negotiations strive for a "win-win" result, stakeholders win by getting a product satisfying most of their needs and developers win by getting achievable deadlines.

Handshaking is one-way to achieve "win-win"

- Developers propose solutions to requirements, describe their impact, and communicate their intentions to the customers.
- Customer review the proposed solutions, focusing on missing features and seeking clarification of novel requirements.
- Requirements are determined to be good enough if the customers accept the proposed solutions.

Handshaking tends to improve identification, analysis, and selection of variants.

Requirements Monitoring

Useful for incremental development includes:

1. **Distributed debugging** - uncovers errors and determines their cause.

2. **Run-time verification** - determines whether software matches its specification.
3. **Run-time validation** - assesses whether the evolving software meets user goals.
4. **Business activity monitoring** - evaluates whether a system satisfies business goals.
5. **Evolution and codesign** - provides information to stakeholders as the system evolves.

Validating requirements

- Is each requirement consistent with the overall objective for the system/product?
- Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
- Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
- Is each requirement bounded and unambiguous?
- Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
- Do any requirements conflict with other requirements?
- Is each requirement achievable in the technical environment that will house the system or product?
- Is each requirement testable, once implemented?
- Does the requirements model properly reflect the information, function and behavior of system to be built?
- Has the requirements model been "partitioned" in a way that exposes progressively more detailed information about the system?
- Have requirements patterns been used to simplify the requirements model. Have all patterns been properly validated?
- Are all patterns consistent with customer requirements?

Week 8

Requirements Engineering

What is a requirement?

It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification.

This is inevitable as requirements may serve a dual function

- May be the basis for a bid for a contract - therefore must be open to interpretation.
- May be the basis for the contract itself - therefore must be defined in detail.
- Both these statements may be called requirements.

Requirements Abstraction (Davis)

"If a company wishes to let a contract for a large software development project, it must define its needs in a sufficiently abstract way that a solution is not pre-defined. The requirements must be written so that several contractors can bid for the contract, offering, perhaps, different ways of meeting the client organization's needs.

Once a contract has been awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do. Both of these documents may be called the requirements document for the system."

Types of Requirements

User requirements

- Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.

System requirements

- A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

User and system requirements

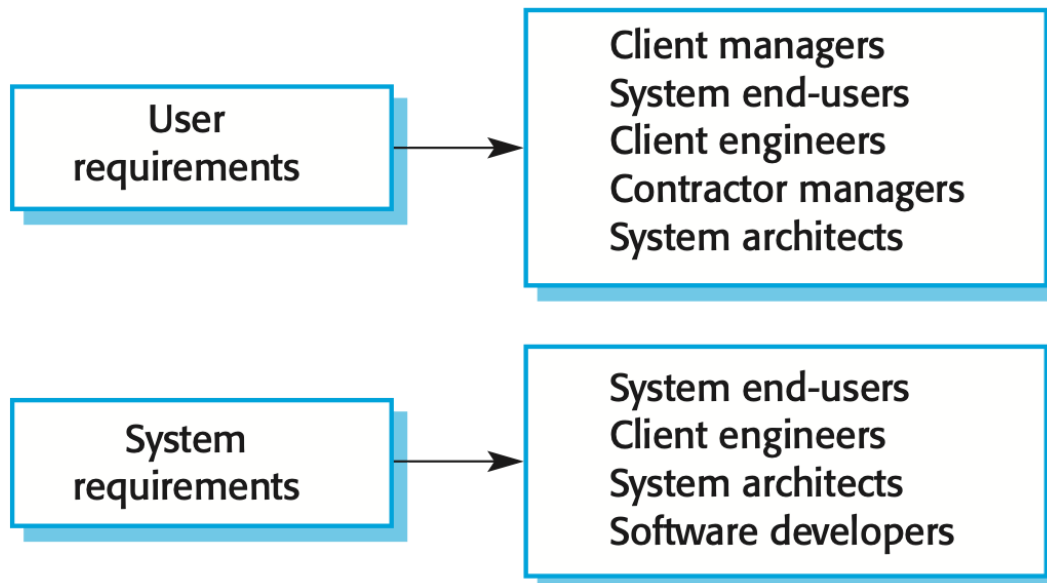
User requirements definition

1. The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2 The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4 If drugs are available in different dose units (e.g. 10mg, 20mg, etc) separate reports shall be created for each dose unit.
- 1.5 Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

Readers of different types of requirements specification



Functional & Non-Functional Requirements

- Functional requirements
 - Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in

particular situations.

- May state what the system should not do.
- Non-functional requirements
 - Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
 - Often apply to the system as a whole rather than individual features or services.
- Domain requirements
 - Constraints on the system from the domain of operation

Functional Requirements

- Describe functionality or system services.
- Depend on the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.

Mentcare System: Functional Requirements

A user shall be able to search the appointments lists for all clinics.

The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.

Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.

Week 9

Prototyping and Requirements Presentations

Prototype Definition & Recap

- Prototyping is the process of quickly putting together a working model (a prototype) in order to test various aspects of a design, illustrate ideas or features and gather early user feedback.
- The IEEE defines prototyping as a development approach promoting the implementation of a pilot version of the intended product.
- Often, a customer defines a set of general objectives for software but does not identify detailed requirements for functions and features. In other cases, the developer may be unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form that human-machine interaction should take. In these, and many other situations, a prototyping paradigm may offer the best approach.

Need for prototyping

- Enables us to explore the problem space with the stakeholders.
- As a requirements artifact to initially envision the system.
- As a design artifact that enables us to explore the solution space of your system.
- A vehicle for you to communicate the possible UI design(s) of your system.
- A potential foundation from which to continue developing the system

Types of Prototypes

Throwaway Prototypes

- Throw Away Prototype is developed from the initial requirements but is not used for the final project.
- Written specifications of the requirements
- Some developers believe that this type is a waste of time because you don't use it.
- Regardless if prototype is discarded or kept for production, you must use a easy to use language.

<u>Advantages</u>	<u>Disadvantages</u>
Significantly reduce project risk	The prototype actually does nothing, its just presentational.
Has a short project timeline	Only for a limited purpose
	Starting become a thing of the past. Not getting used as much now.

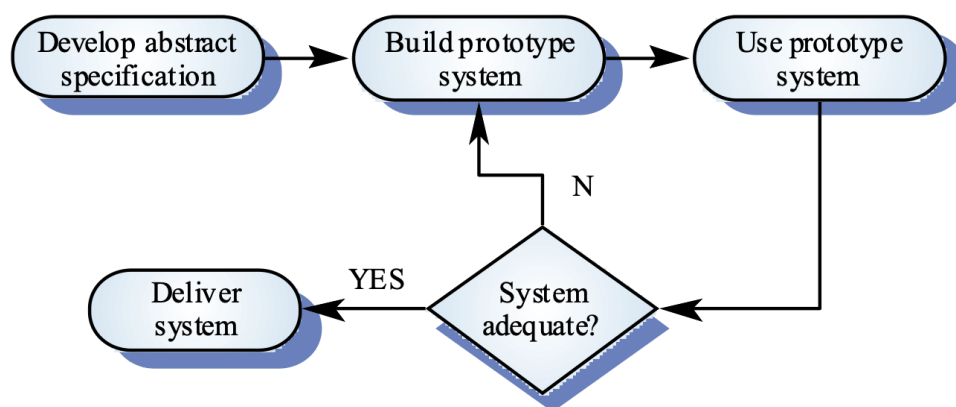
Evolutionary Prototypes

Evolutionary prototyping is consider the most fundamental form of prototyping.

Evolutionary prototyping main concept is to build a robust prototype and constantly improve it.

Objective to deliver a working system to the end user.

According to Steve McConnell, "evolutionary delivery is a lifecycle model that straddles the ground between evolutionary prototyping and staged delivery."



Advantages

You are always looking for new ways to improve the system.

This model increases the chance of having the client satisfied with the working system.

The model can be used even when the requirements are not defined.

Quicker delivery of the system

Disadvantages

This method can be used to avoid documenting the requirements of the system.

Management is required

Long term maintenance can be expensive

Information can be lost through so many improvement changes

Low Fidelity Prototype

Low-fidelity prototyping is generally limited function, limited interaction prototyping effort.

They are constructed to depict concepts, design alternatives and screen layouts. They are intended to demonstrate general look and feel of the interface.

They are created to educate , communicate and inform, but not to train, test or serve as a basis for which to code.

Low fidelity prototyping is used early in the design cycle to show general conceptual approaches without much investment in development.

High Fidelity Prototypes

High-fidelity prototypes represent the core functionality of the products user interface.

High fidelity prototypes are fully interactive systems. Users can enter data in entry fields, respond to messages, select icon to open windows and interact with user interface as if it were a real system.

They trade-off speed for accuracy.

Building high fidelity prototypes consume resources and have high cost.

Comparison between High & Low Fidelity Prototypes

Type	Advantages	Disadvantages
Low-fidelity Prototypes	• Lower Development cost • Evaluate multiple design concept • Useful communication device • Address screen layout issues • Address screen layout issues • Useful for identifying requirements	• Limited error checking • Facilitator driven • Limited utility after requirements established • Navigational and flow limitations • Poor detailed specifications to code
High-fidelity Prototypes	• Complete functionality • Fully interactive • User Driven • Clear definition of navigation • Look and feel of final product • Use for explorations and tests	• More expensive to develop • Time-consuming to create • Not effective for requirements gathering

User Interface Prototyping

Development of highly interactive software system with graphical user interfaces (GUI) has become increasingly common and acceptance of such a system depends highly on the quality of GUI.

Prototyping is an excellent means of generating ideas about how the GUI can be designed and it helps to evaluate quality of solution at an early stage.

Classifications of User Interface Prototypes

Presentation Prototypes

Functional Prototypes

Breadboards

Pilot Systems

Presenting Designs in a Meeting

If presenting to Senior Management:

- Vision
- Concepts
- Key features
- Impact
- Image

If presenting to client:

- Detail of design
- Workings of system

If aimed at getting a contract:

- The main selling point
- Differences between your product and others

If aim is to agree a concept:

- Restate client brief
- Clarify requirements
- Scope the project

After Prototype Has Been Used

Usually the best choice is to throw the prototype away

It is meant to be used as a sketch

There are many real-world examples of failures, when code parts of prototypes have been used in products

Prototypes, which 'look too good' can be potentially dangerous:

- Customer may think, that the product is almost finished
- Management is not willing to throw 'almost finished' parts of prototype away

One way of avoiding prototype's code to be used is to implement prototype with a language unsuitable for the product

Requirements Presentation

Requirements analysis results in the specification of software's operational characteristics, indicates software's interface with other system elements, and establishes constraints that software must meet.

Requirements analysis allows you (regardless of whether you're called a software engineer, an analyst, or a modeler) to elaborate on basic requirements established during the inception, elicitation, and negotiation tasks that are part of requirements engineering.

The requirements modeling action results in one or more of the following types of:

- Scenario-based models of requirements from the point of view of various System "actors."
- Class-oriented models that represent object-oriented classes (attributes and operations) and how classes collaborate to achieve system requirements.
- Behavioral models that depict how the software reacts to internal or external "events."
- Data models that depict the information domain for the problem.
- Flow-oriented models that represent the functional elements of the system and how they transform data as they move through the system.

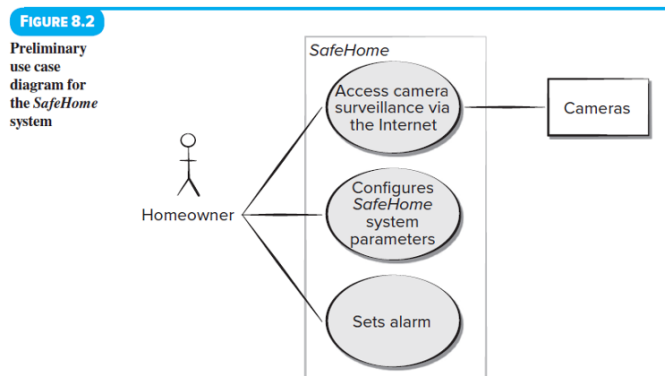
These models provide a software designer with information that can be translated to architectural interface, and component-level designs. Finally, the requirements model (and the software requirements specification) provides the developer and the customer with the means to assess quality once software is built.

Scenario-Based Models

Although the success of a computer-based system or product is measured in many ways, user satisfaction resides at the top of the list.

If you understand how end users (and other actors) want to interact with a system, your software team will be better able to properly characterize requirements and build meaningful analysis and design models.

Using UML to model requirements begins with the creation of scenarios in the form of use case diagrams, activity diagrams, and sequence diagrams.



Class-Oriented Models

If you look around a room, there is a set of physical objects that can be easily identified, classified, and defined (in terms of attributes and operations). But when you “look around” the problem space of a software application, the classes (and objects) may be more difficult to comprehend. Class-responsibility-collaborator (CRC) modeling [Wir90] provides a simple means for identifying and organizing the classes that are relevant to system or product requirements. A CRC model can be viewed as a collection of index cards.

FIGURE 8.5

A CRC model index card

Class: FloorPlan	
Description	
Responsibility:	Collaborator:
Defines floor plan name/type	
Manages floor plan positioning	
Scales floor plan for display	
Incorporates walls, doors, and windows	Wall
Shows position of video cameras	Camera

Behavioral Models

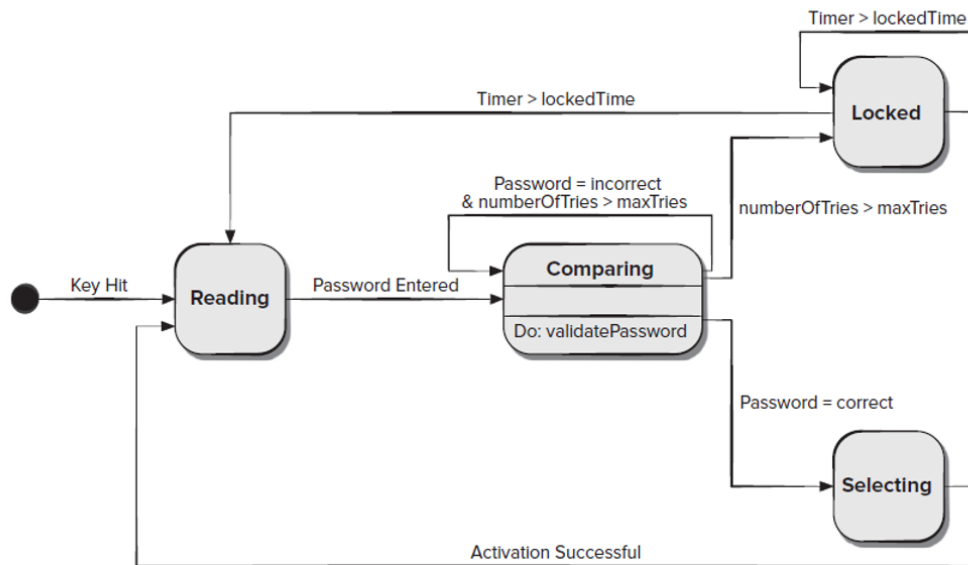
A behavioral model indicates how software will respond to internal or external events. This information is useful in the creation of an effective design for the system to be built. UML activity diagrams can be used to model how system elements respond to internal events. UML state diagrams can be used to model how system elements respond to external events.

To create the model, you should perform the following steps:

1. Evaluate all use cases to fully understand the sequence of interaction within the system
2. Identify events that drive the interaction sequence and understand how these events relate to specific objects
3. Create a sequence for each use case
4. Build a state diagram for the system
5. Review the behavioral model to verify accuracy and consistency. Each of these steps is discussed in the sections that follow

State Diagrams

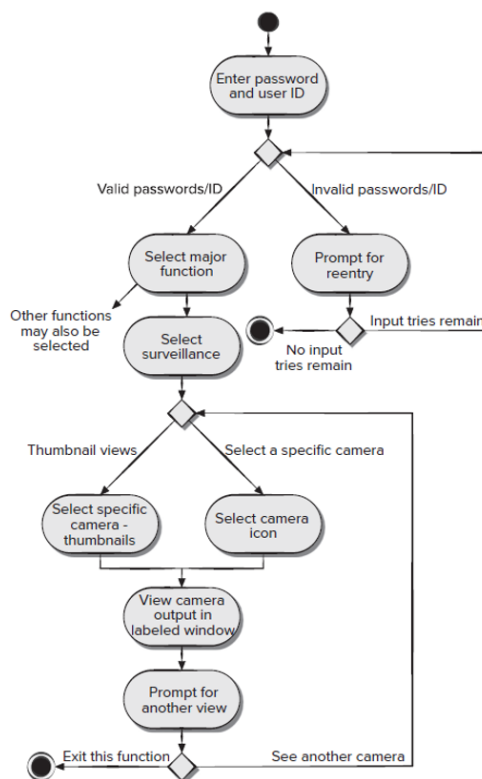
FIGURE 8.8 State diagram for the ControlPanel class



Activity Diagrams

FIGURE 8.9

Activity diagram for Access camera surveillance via the Internet—display camera views function



Week 10

Software Analysis & Design

System Analysis

- **Systems analysis** defines the problems to be solved and provides the architecture of the proposed system.
- The terms **analysis** and **synthesis** come from Greek where they mean respectively "to take apart" and "to put together".
- **Analysis** is defined as the procedure by which we break down an intellectual or substantial whole into parts.
- **Synthesis** is defined as the procedure by which we combine separate elements or components in order to form a coherent whole.
- System analysis is an explicit formal inquiry carried out to help a decision maker identify a better course of action and make a better decision than he might otherwise have made.
- Systems analysis is a problem-solving technique that decomposes a system into its component pieces for the purpose of studying how well those component parts work and interact to accomplish their purpose.
- This is a process used in the design of new systems. Systems analysis follows stages of investigation, design and implementation.
- Each stage should involve close consultation with potential users, in the various functional areas of the organisation, to ensure that their information and operational requirements are met.

When to use system analysis and design

- To correct problem in existing system
- To improve existing system
- Usher in a new system
- Outside group may mandate change
- Competition can lead to change

Software Design

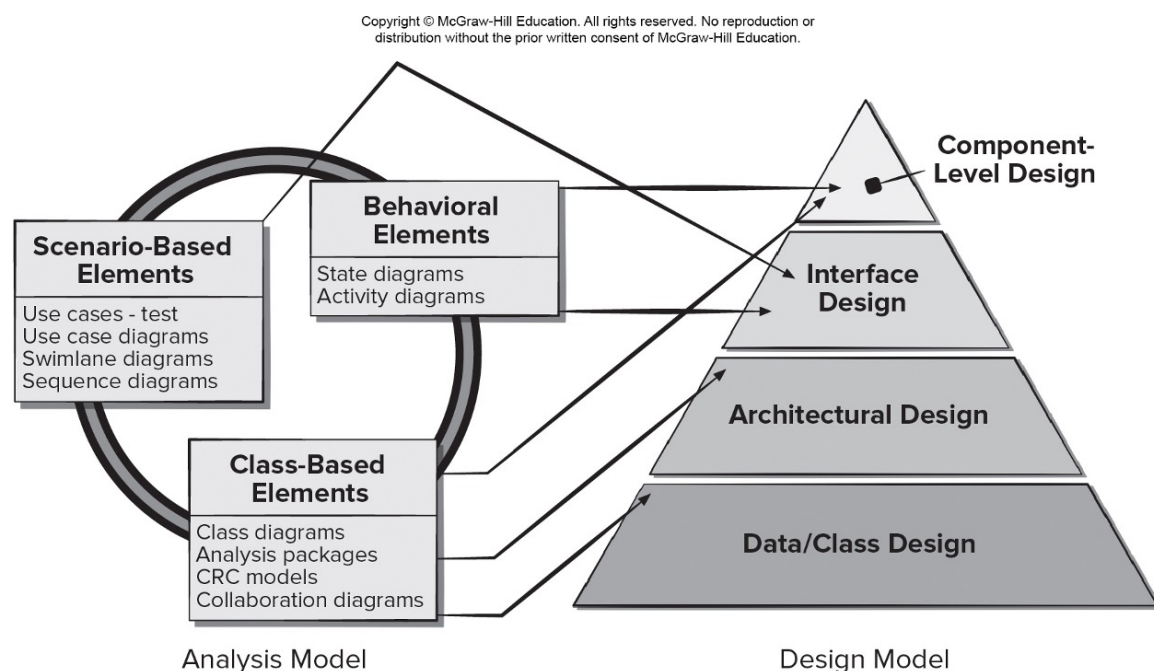
- Encompasses the set of principles, concepts, and practices that lead to the development of a high quality system or product.

- Design principles establish an overriding philosophy that guides the designer as the work is performed.
- Design concepts must be understood before the mechanics of design practice are applied.
- Software design practices change continuously as new methods, better analysis, and broader understanding evolve.

Software Engineering Design

- **Data/Class design** – transforms analysis classes into implementation classes and data structures.
- **Architectural design** – defines relationships among the major software structural elements.
- **Interface design** – defines how software elements, hardware elements, and end-users communicate.
- **Component-level design** – transforms structural elements into procedural descriptions of software components.

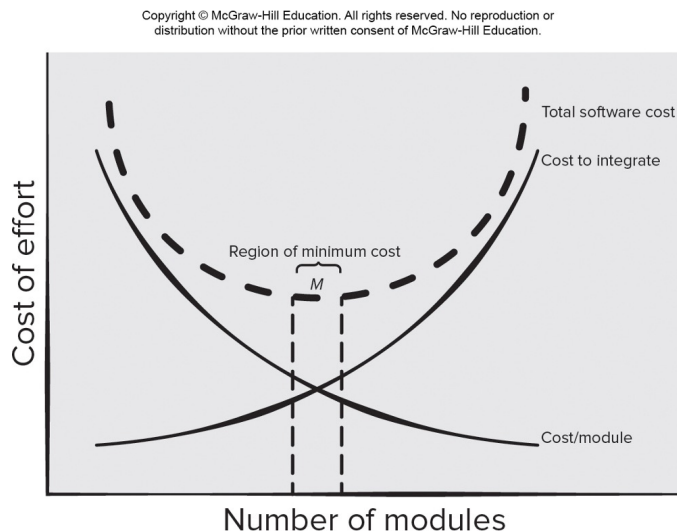
Mapping Requirements Model to Design Model



Design Concepts

- **Abstraction** – data (named collection of data describing data object), procedural (name sequence of instructions with specific and limited function).
- **Architecture** - overall structure or organization of software components, ways components interact, and structure of data used by components.
- **Design Patterns** - describe a design structure that solves a well-defined design problem within a specific context.
- **Separation of concerns** - any complex problem can be more easily handled if it is subdivided into pieces.
- **Modularity**—compartmentalization of data and function.

Modularity and Software Cost

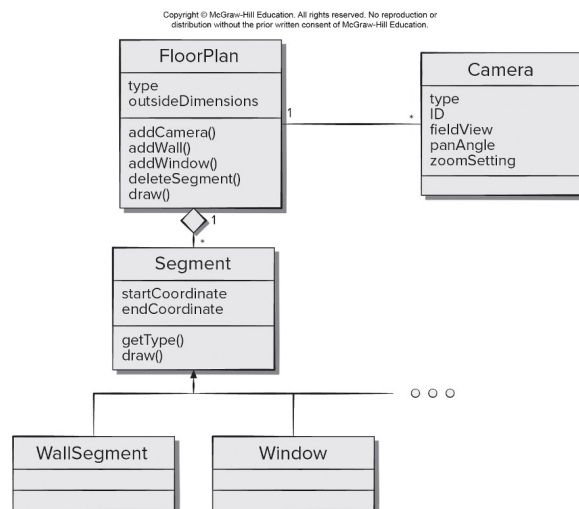


Design Concepts

- **Information Hiding** - controlled interfaces which define and enforces access to component procedural detail and any local data structure used by the component.
- **Functional independence** - single-minded (high cohesion) components with aversion to excessive interaction with other components (low coupling).
- **Stepwise Refinement** – incremental elaboration of detail for all abstractions.

- **Refactoring**—a reorganization technique that simplifies the design without changing functionality.
- **Design Classes**—provide design detail that will enable analysis classes to be implemented.

Design Class Example



Design Class Characteristics

- **Complete** - includes all necessary attributes and methods) and sufficient (contains only those methods needed to achieve class intent).
- **Primitiveness** – each class method focuses on providing one service.
- **High cohesion** – small, focused, single-minded classes.
- **Low coupling** – class collaboration kept to minimum.

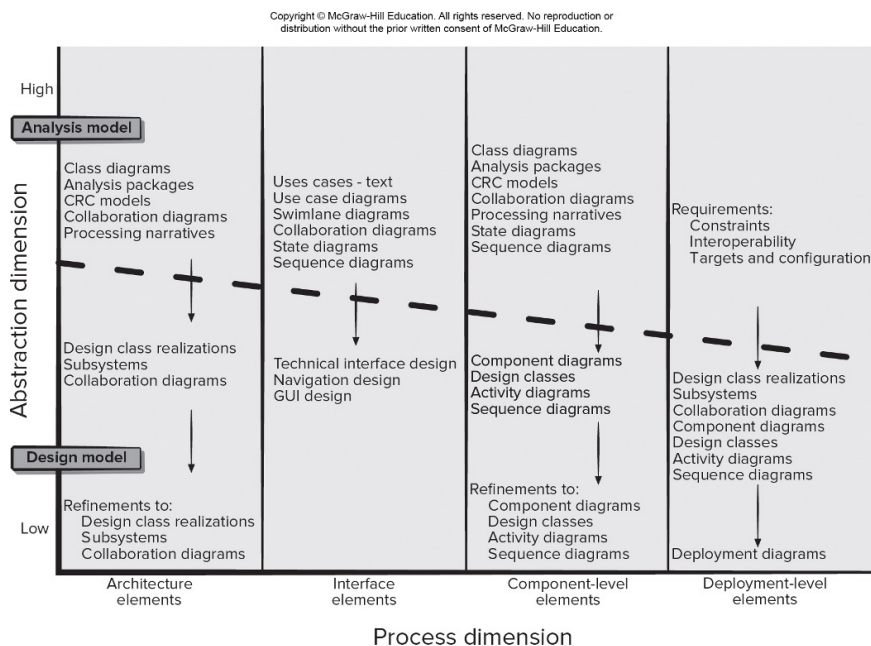
Information Hiding

- Reduces the likelihood of "side effects."
- Limits the global impact of local design decisions.
- Emphasizes communication through controlled interfaces.
- Discourages the use of global data. Hard to accomodate changes.
- Leads to encapsulation—an attribute of high quality design.
- Results in higher quality software.

Design Pattern Template

- **Pattern name** - describes the essence of the pattern in a short but expressive name
- **Intent** - describes the pattern and what it does
- **Also-known-as** - lists any synonyms for the pattern
- **Motivation** - provides an example of the problem
- **Applicability** - notes specific design situations in which the pattern is applicable
- **Structure** - describes the classes that are required to implement the pattern
- **Participants** - describes the responsibilities of the classes that are required to implement the pattern
- **Collaborations** - describes how the participants collaborate to carry out their responsibilities
- **Consequences** - describes the "design forces" that affect the pattern and the potential trade-offs that must be considered when the pattern is implemented
- **Related patterns** - cross-references related design patterns

Design Model



Low abstraction means more details remains whereas High abstraction means lesser details remain.

Data Design Elements

Data model – data objects and database architectures.

- Examines data objects independently of processing.
 - Focuses attention on the data domain.
 - Creates a model at the customer's level of abstraction.
 - Indicates how data objects relate to one another.
1. Data object can be an external entity, a thing, an event, a place, a role, an organizational unit, or a structure.
 2. Data objects contain a set of attributes that act as an quality, characteristic, or descriptor of the object.
 3. Data objects may be connected to one another in many different ways.
 4. Have methods. Methods are also known as behavior and

Architectural Design Elements

Architectural design for software - equivalent to the floor plan for a house.

The architectural model is derived from three sources:

- Information about the application domain for the software to be built.
- Specific requirements model elements such as data flow analysis classes and their relationships (collaborations) for the problem at hand, and
- Availability of architectural patterns and styles.
- Also talks about database.

Interface Design Elements

Interface is a set of operations that describes the externally observable behavior of a class and provides access to its public operations.

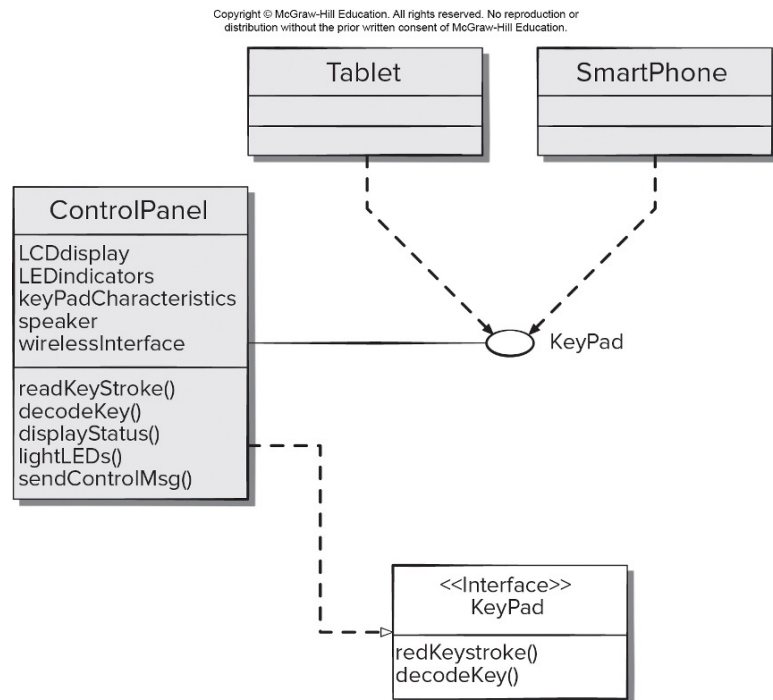
Important elements:

- User interface (UI) or graphical user interface (GUI).
- External interfaces to other systems.
- Internal interfaces between various design components.

UI or User Experience (UX) is a major engineering action to ensure the creation on usable software products.

Internal and external interfaces should incorporate both **error checking** and appropriate **security features**.

Interface Model for Control Panel



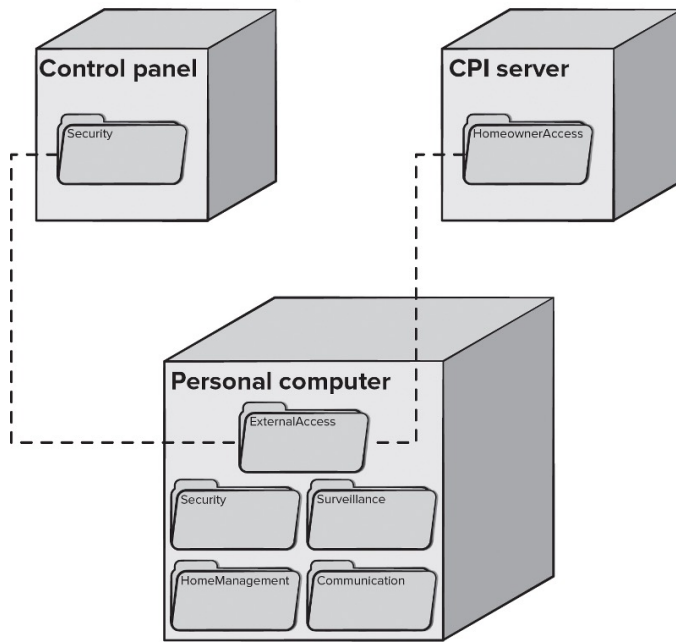
Deployment Design Elements

Indicates how software functionality and subsystems will be allocated within the physical computing environment.

Modeled using UML deployment diagrams.

- **Descriptor form deployment diagrams** - show the computing environment but does not indicate configuration details.
- **Instance form deployment diagrams** - identify specific hardware configurations and are developed in the latter stages of design.

UML Deployment Instance Diagram



Week 11

Object Oriented Design

Design and Implementation

Software design and implementation is the stage in the software engineering process at which an executable software system is developed.

Software design and implementation activities are invariably inter-leaved.

- Software design is a creative activity in which you identify software components/features and their relationships, based on a customer's requirements.
- Implementation is the process of realizing the design as a program.

An object oriented design process

- Structured object-oriented design processes involve developing a number of different system models.
- They require a lot of effort for development and maintenance of these models and, for small systems, this may not be cost-effective.
- However, for large systems developed by **different groups** design models are an important communication mechanism.

Process Stages

- There are a variety of different object-oriented design processes that depend on the organization using the process.
- Common activities in these processes include:
 - Define the context and modes of use of the system;
 - Design the system architecture;
 - Identify the principal system objects/entities;
 - Develop design models;
 - Specify object interfaces.
- Process illustrated here using a design for a wilderness weather station.

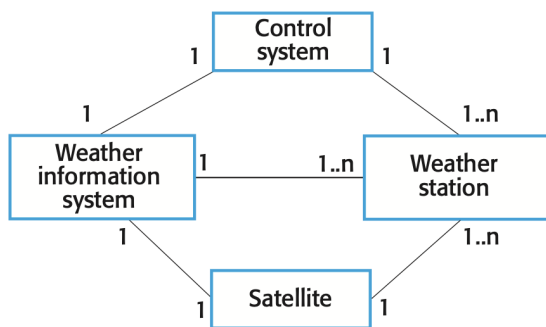
System Context & Interactions

- Understanding the relationships between the software that is being designed and its external environment is essential for deciding how to provide the required system functionality and how to structure the system to communicate with its environment.
- Understanding of the context also lets you establish the boundaries/user interfaces of the system. Setting the system boundaries helps you decide what features are implemented in the system being designed and what features are in other associated systems.

Context & Interaction Models

- A system context model is a structural model that demonstrates the other systems in the environment of the system being developed.
- An interaction model is a dynamic model that shows how the system interacts with its environment as it is used. (Example: Use Case Diagram)

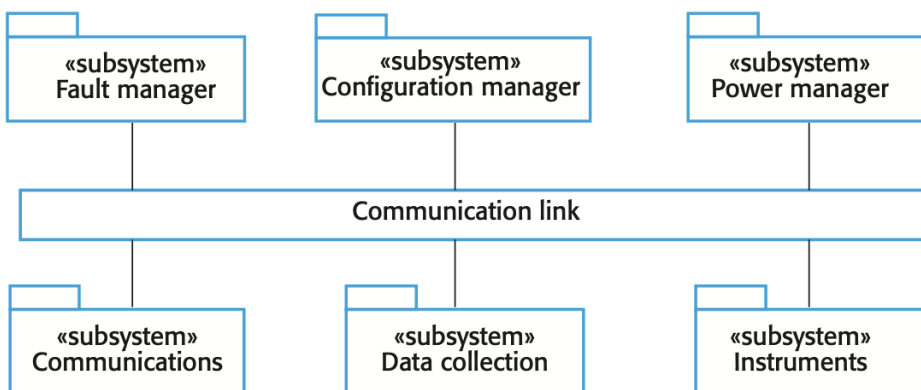
System Context for Weather Station



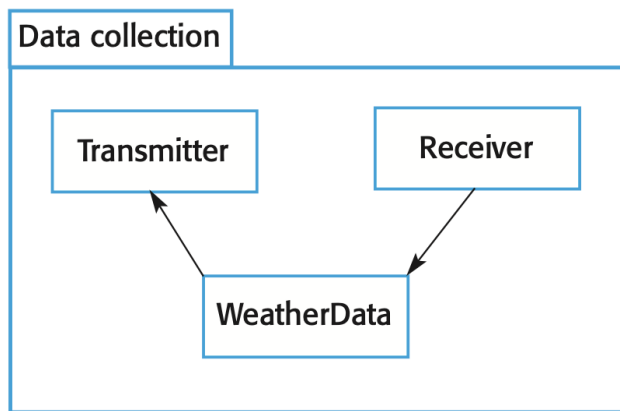
Architectural Design

- Once interactions between the system and its environment have been understood, you use this information for designing the system architecture.
- You identify the major components that make up the system and their interactions, and then may organize the components using an architectural pattern such as a layered or client-server model.
- The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure.

High-level Architecture of the Weather Station



Architecture of Data Collection System



Basic Component Design Principles for Class Diagram (Not Tested)

- **Open-Closed Principle (O C P).** "A module [component] should be open for extension but closed for modification."
- **Liskov Substitution Principle (L S P).** "Subclasses should be substitutable for their base classes."
- **Dependency Inversion Principle (D I P).** "Depend on abstractions. Do not depend on concretions."
- **Interface Segregation Principle (I S P).** "Many client-specific interfaces are better than one general purpose interface."
- **Release Reuse Equivalency Principle (R E P).** "The granule of reuse is the granule of release."
- **Common Closure Principle (C C P).** "Classes that change together belong together."
- **Common Reuse Principle (C R P).** "Classes that aren't reused together should not be grouped together."

Component Level Design Guidelines

- **Components** - Naming conventions should be established for components that are specified as part of the architectural model and then refined and elaborated as part of the component-level model.
- **Interfaces** - provide important information about communication and collaboration (as well as helping us to achieve the O P C).
- **Dependencies and Inheritance** – For readability, it is a good idea to model dependencies from left to right and inheritance from bottom (derived

classes) to top (base classes).

Cohesion

- **Traditional view** - the "single-mindedness" of a module.
- **Object-Oriented view** - cohesion implies that a component encapsulates only attributes and operations that are closely related to one another and the component itself.
- Levels of cohesion:
 - **Functional** - module performs one and only one computation.
 - **Layer** - occurs when a higher layer accesses the services of a lower layer, but lower layers do not access higher layers.
 - **Communicational** - All operations that access the same data are defined within one class.

Coupling

- **Traditional view** - degree to which a component is connected to other components and to the external world.
- **Object-Oriented view** - qualitative measure of the degree to which classes are connected to one another.
- Levels of coupling.
 - **Content** - occurs when one component secretly modifies data that is internal to another component.
 - **Control** - occurs when control flags are passed to components to request alternate behaviors when invoked.
 - **External** - occurs when a component communicates or collaborates with infrastructure components.

Object Class Identification

- Identifying object classes is often a difficult part of object oriented design.
- There is no 'magic formula' for object identification. It relies on the skill, experience and domain knowledge of system designers.
- Object identification is an iterative process. You are unlikely to get it right first time.

Approaches to Identification

- Use a grammatical approach based on a natural language description of the system.
- Base the identification on tangible things in the application domain.
- Use a behavioural approach and identify objects based on what participates in what behaviour.
- Use a scenario-based analysis. The objects, attributes and methods in each scenario are identified.

Weather Station Object Classes

- Object class identification in the weather station system may be based on the tangible hardware and data in the system:
 - Ground thermometer, Anemometer, Barometer
 - Application domain objects that are 'hardware' objects related to the instruments in the system.
 - Weather station
 - The basic interface of the weather station to its environment. It therefore reflects the interactions identified in the use-case model.
 - Weather data
 - Encapsulates the summarized data from the instruments.

```
reportWeather ( )
reportStatus ( )
powerSave (instruments)
remoteControl (commands)
reconfigure (commands)
restart (instruments)
shutdown (instruments)
```

```
groundTemperatures
windSpeeds
windDirections
pressures
rainfall

collect ( )
summarize ( )
```

Ground thermometer
gt_Ident temperature

Anemometer
an_Ident windSpeed windDirection

Barometer
bar_Ident pressure height

Design Models

- Design models show the objects and object classes and relationships between these entities.
- There are two kinds of design model:
 - Structural models describe the static structure of the system in terms of object classes and relationships.
 - Dynamic models describe the dynamic interactions between objects.

Examples of Design Model

- Subsystem models that show logical groupings of objects into coherent subsystems.
- Sequence models that show the sequence of object interactions.
- State machine models that show how individual objects change their state in response to events.
- Other models include use-case models, aggregation models, generalisation models, etc.

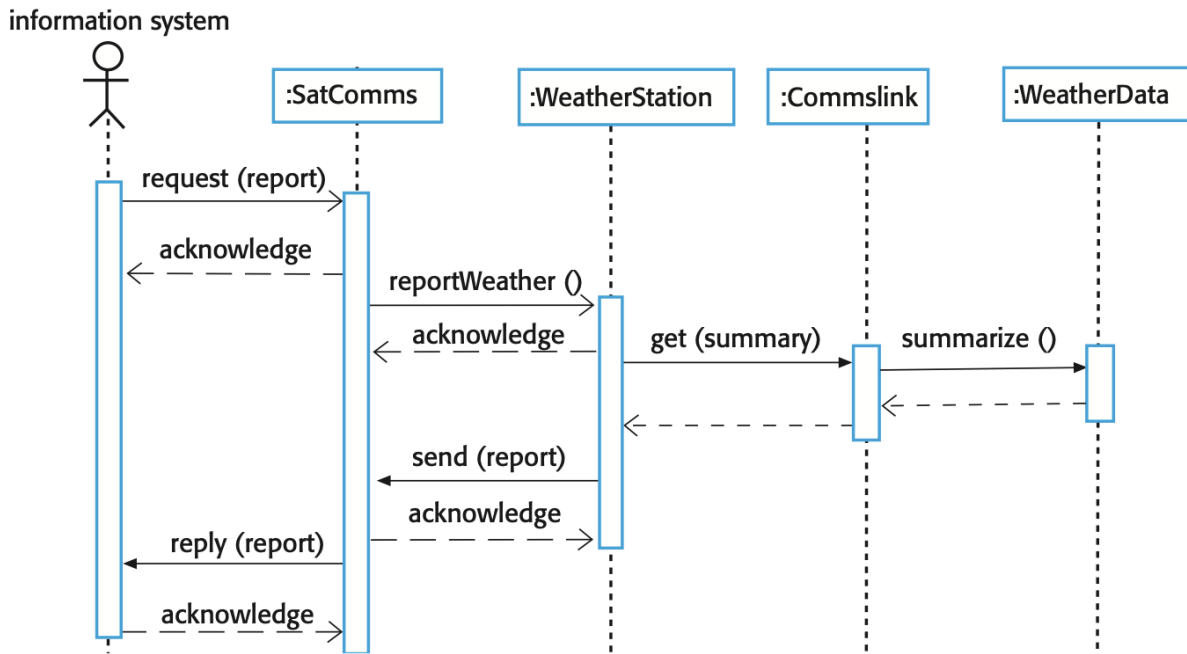
Subsystem Models

- Shows how the design is organised into logically related groups of objects.
- In the UML, these are shown using packages - an encapsulation construct. This is a logical model. The actual organisation of objects in the system may be different.

Sequence Models

- Sequence models show the sequence of object interactions that take place
 - Objects are arranged horizontally across the top;
 - Time is represented vertically so models are read top to bottom;
 - Interactions are represented by labelled arrows, Different styles of arrow represent different types of interaction;
 - A thin rectangle in an object lifeline represents the time when the object is the controlling object in the system.

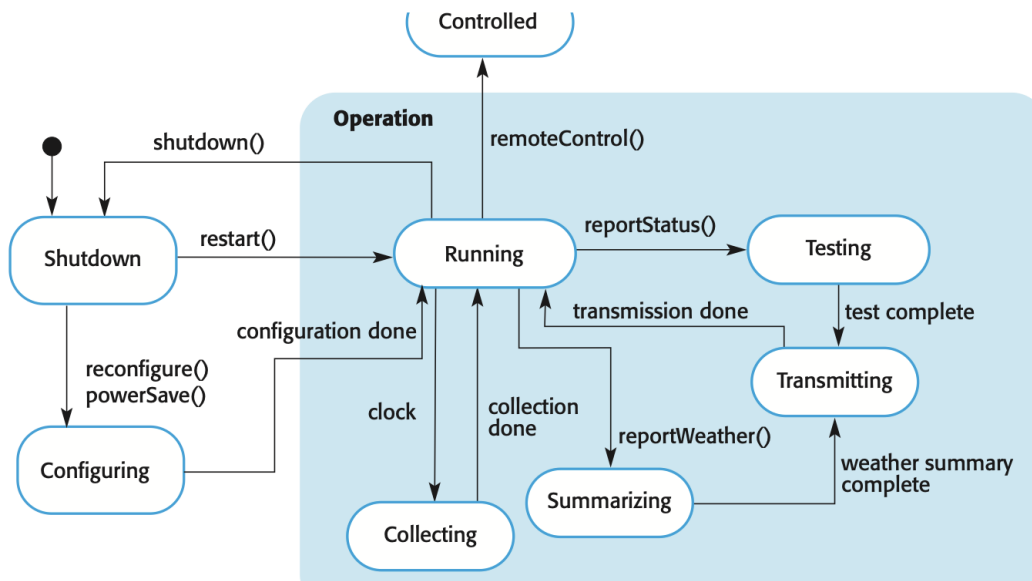
Sequence Diagram Describing Data Collection



State Diagrams

- State diagrams are used to show how objects respond to different service requests and the state transitions triggered by these requests.
- State diagrams are useful high-level models of a system or an object's run-time behavior.
- You don't usually need a state diagram for all of the objects in the system. Many of the objects in a system are relatively simple and a state model adds unnecessary detail to the design.

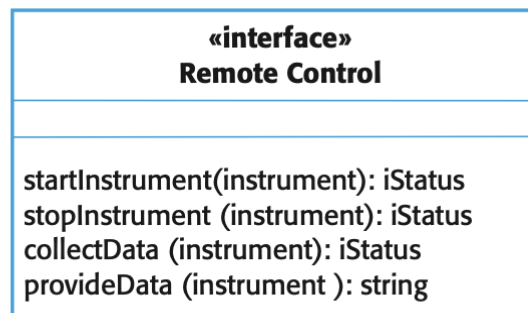
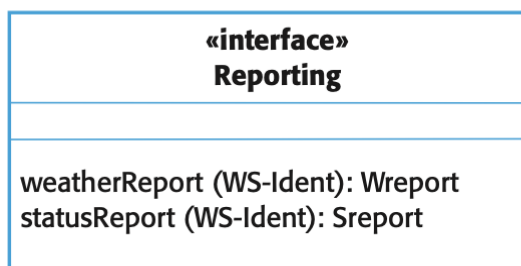
Weather Station State Diagram



Interface Specification

- Object interfaces have to be specified so that the objects and other components can be designed in parallel.
- Designers should avoid designing the interface representation but should hide this in the object itself.
- Objects may have several interfaces which are viewpoints on the methods provided.
- The UML uses class diagrams for interface specification but Java may also be used.

Weather Station Interfaces



Design Patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution.
- A pattern is a description of the problem and the essence of its solution.
- It should be sufficiently abstract to be reused in different settings.
- Pattern descriptions usually make use of object-oriented characteristics such as inheritance and polymorphism.

Patterns and Pattern Languages are ways to describe best practices, good designs, and capture experience in a way that it is possible for others to reuse this experience.

Pattern Elements

- Name
 - A meaningful pattern identifier.
- Problem description.
- Solution description.
 - Not a concrete design but a template for a design solution that can be instantiated in different ways.
- Consequences
 - The results and trade-offs of applying the pattern.

The Observer Pattern

- Name
 - Observer.
- Description
 - Separates the display of object state from the object itself.
- Problem description
 - Used when multiple displays of state are needed.
- Solution description
 - See slide with UML description.
- Consequences

- Optimisations to enhance display performance are impractical.

Design Problems

- To use patterns in your design, you need to recognize that any design problem you are facing may have an associated pattern that can be applied.
 - Tell several objects that the state of some other object has changed (**Observer pattern**).
 - Tidy up the interfaces to a number of related objects that have often been developed incrementally (**Façade pattern**).
 - Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (**Iterator pattern**).
 - Allow for the possibility of extending the functionality of an existing class at run-time (**Decorator pattern**).

Week 12

Quality and Security

Strategic Approach to Testing

- You should conduct effective technical reviews this can eliminate many errors before testing begins.
- Testing begins at the component level and works "outward" toward the integration of the entire system.
- Different testing techniques are appropriate for different software engineering approaches and at different points in time.
- Testing is conducted by the developer of the software and (for large projects) an independent test group.
- Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

Verification & Validation

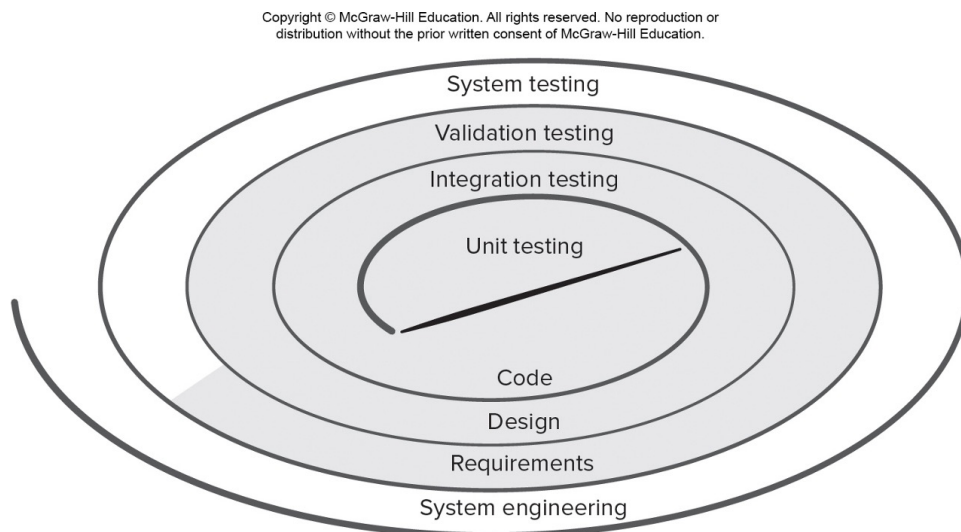
- **Verification** refers to the set of tasks that ensure that software correctly implements a specific function.

- Verification: Are we building the product right?
- **Validation** refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements.
 - Validation: "Are we building the right product?"

Organizing for Testing

- Software developers are always responsible for testing individual program components and ensuring that each performs its deigned function or behavior.
- Only after the software architecture is complete does an independent test group become involved.
- The role of an **independent test group** (ITG) is to remove the inherent problems associated with letting the builder test the thing that has been built.
- ITG personnel are paid to find errors.
- Developers and ITG work closely throughout a software project to ensure that thorough tests will be conducted.

Testing Strategy



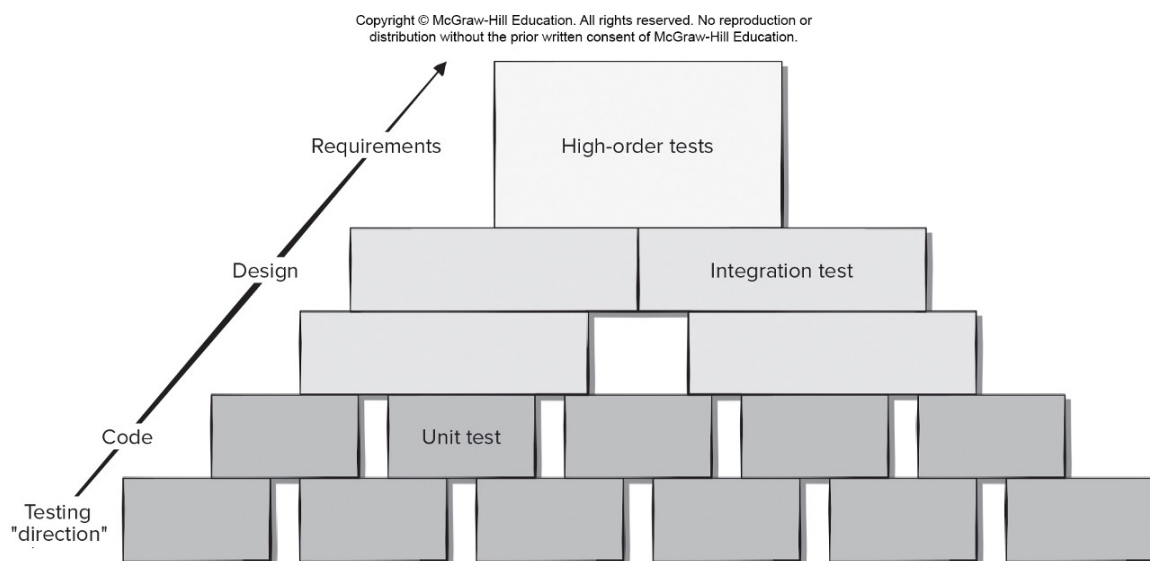
Testing the Big Picture

- Unit testing begins at the center of the spiral and concentrates on each unit (for example, component, class, or content object) as they are implemented

in source code.

- Testing progresses to integration testing, where the focus is on design and the construction of the software architecture. Taking another turn outward on the spiral.
- Validation testing, is where requirements established as part of requirements modeling are validated against the software that has been constructed.
- In system testing, the software and other system elements are tested as a whole.

Software Testing Steps



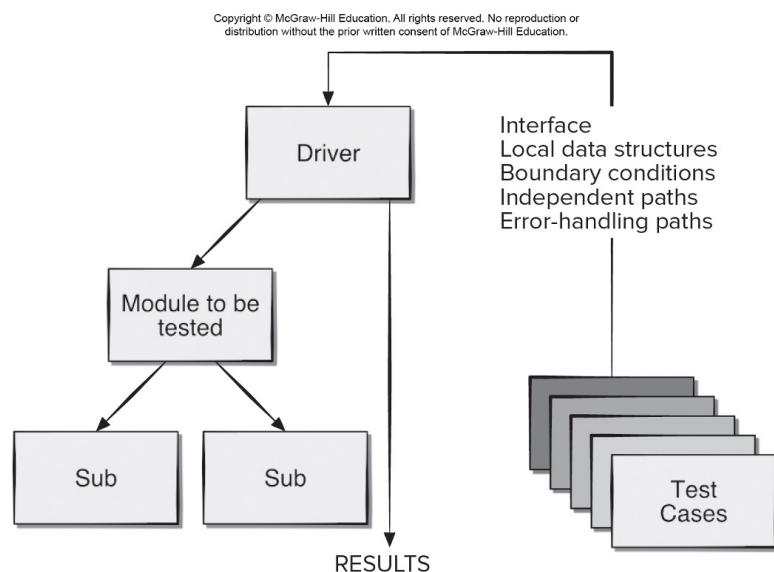
When is Testing Done?

- You're never done testing; the burden simply shifts from the software engineer to the end user. (Wrong).
- You're done testing when you run out of time or you run out of money. (Wrong).
- The statistical quality assurance approach suggests executing tests derived from a statistical sample of all possible program executions by all targeted users.
- By collecting metrics during software testing and making use of existing statistical models, it is possible to develop meaningful guidelines for answering the question: "When are we done testing?"

Plan Testing

1. Specify product requirements in a quantifiable manner long before testing commences.
2. State testing objectives explicitly.
3. Understand the users of the software and develop a profile for each user category.
4. Develop a testing plan that emphasizes "rapid cycle testing."
5. Build "robust" software that is designed to test itself.
6. Use effective technical reviews as a filter prior to testing.
7. Conduct technical reviews to assess the test strategy and test cases themselves.
8. Develop a continuous improvement approach for the testing process.

Unit Test Environment



Cost Effective Testing

- Exhaustive testing requires every possible combination and ordering of input values be processed by the test component.
- The return on exhaustive testing is often not worth the effort, since testing alone cannot be used to prove a component is correctly implemented.

- Testers should work smarter and allocate their testing resources on modules crucial to the success of the project or those that are suspected to be error-prone as the focus of their unit testing.

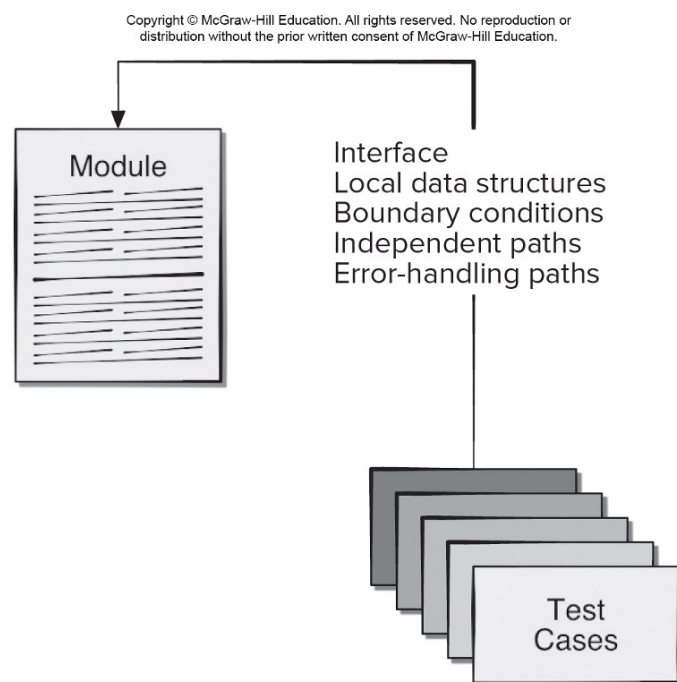
Test Case Design

Design unit test cases before you develop code for a component to ensure that code that will pass the tests.

Test cases are designed to cover the following areas:

- The module interface is tested to ensure that information properly flows into and out of the program unit.
- Local data structures are examined to ensure that stored data stored maintains its integrity during execution.
- Independent paths through control structures are exercised to ensure **all statements are executed at least once**.
- Boundary conditions are tested to ensure module operates properly at boundaries established to limit or restrict processing.
- All error-handling paths are tested.

Module Tests



Error Handling

- A good design anticipates error conditions and establishes error-handling paths which must be tested.
- Among the potential errors that should be tested when error handling is evaluated are:
 1. Error description is unintelligible.
 2. Error noted does not correspond to error encountered.
 3. Error condition causes system intervention prior to error handling,
 4. Exception-condition processing is incorrect.
 5. Error description does not provide enough information to assist in the location of the cause of the error.

Traceability

- To ensure that the testing process is auditable, each test case needs to be traceable back to specific functional or nonfunctional requirements or **anti-requirements**.
- Often nonfunctional requirements need to be traceable to specific business or architectural requirements.
- Many test process failures can be traced to missing traceability paths, inconsistent test data, or incomplete test coverage.
- **Regression testing** requires retesting selected components that may be affected by changes made to other collaborating software components. Deals with changes.

White Box Integration Testing - Structural Testing

- **White-box testing**, is an integration testing philosophy that uses implementation knowledge of the control structures described as part of component-level design to derive test cases.
- **White-box tests can be only be designed after source code exists and program logic details are known.**
- Logical paths through the software and collaborations between components are the focus of white-box integration testing.
- Important data structures should also be tested for validity after component integration.

White Box Testing

Using white-box testing methods, you can derive test cases that:

1. Guarantee that all independent paths within a module have been exercised at least once.
2. Exercise all logical decisions on their true and false sides.
3. Execute all loops at their boundaries and within their operational bounds.
4. Exercise internal data structures to ensure their validity.

Basis Path Testing

Determine the number of independent paths in the program by computing Cyclomatic Complexity:

1. The number of regions of the flow graph corresponds to the cyclomatic complexity.
2. Cyclomatic complexity $V(G)$ for a flow graph G is defined as

$$V(G) = E - N + 2$$

E is the number of flow graph edges

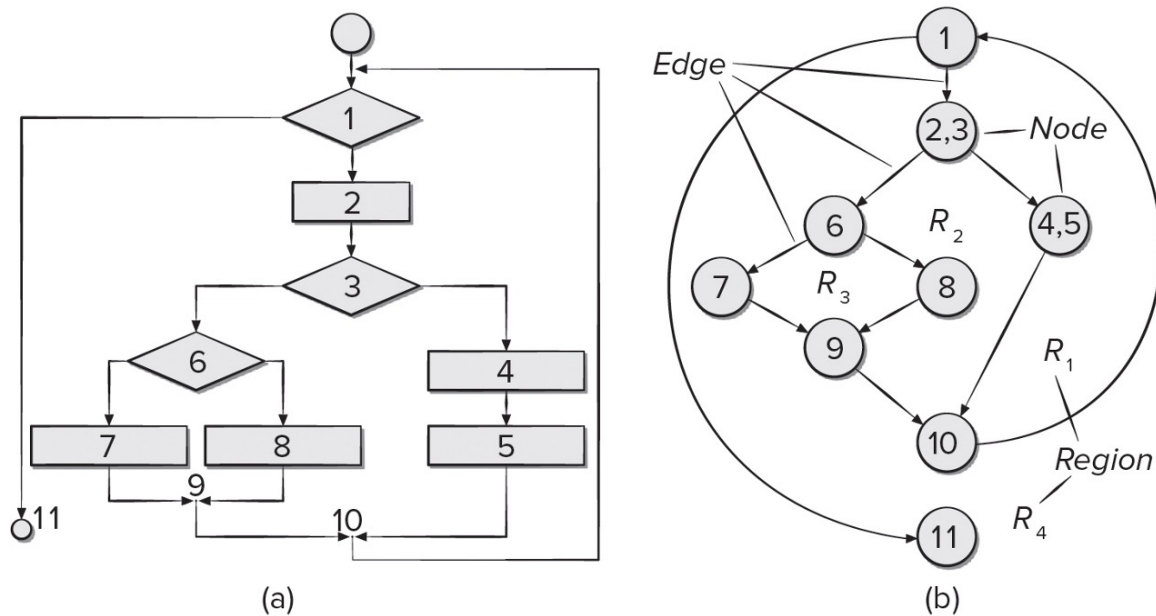
N is the number of nodes.

3. Cyclomatic complexity $V(G)$ for a flow graph G is also defined as

$$V(G) = P + 1$$

P is number of predicate nodes contained in the flow graph G .

Flowchart a and Flow Graph b



Basis Path Testing

Cyclomatic Complexity of the flow graph is 4

1. The flow graph has four regions.
2. $V(G) = 11 \text{ edges} - 9 \text{ nodes} + 2 = 4$.
3. $V(G) = 3 \text{ predicate nodes} + 1 = 4$.

An **independent path** is any path through the program that introduces at least one new set of processing statements or a new condition (we need 4 independent paths to test)

Path 1: 1-11

Path 2: 1-2-3-4-5-10-1-11

Path 3: 1-2-3-6-8-9-10-1-11

Path 4: 1-2-3-6-7-9-10-1-11

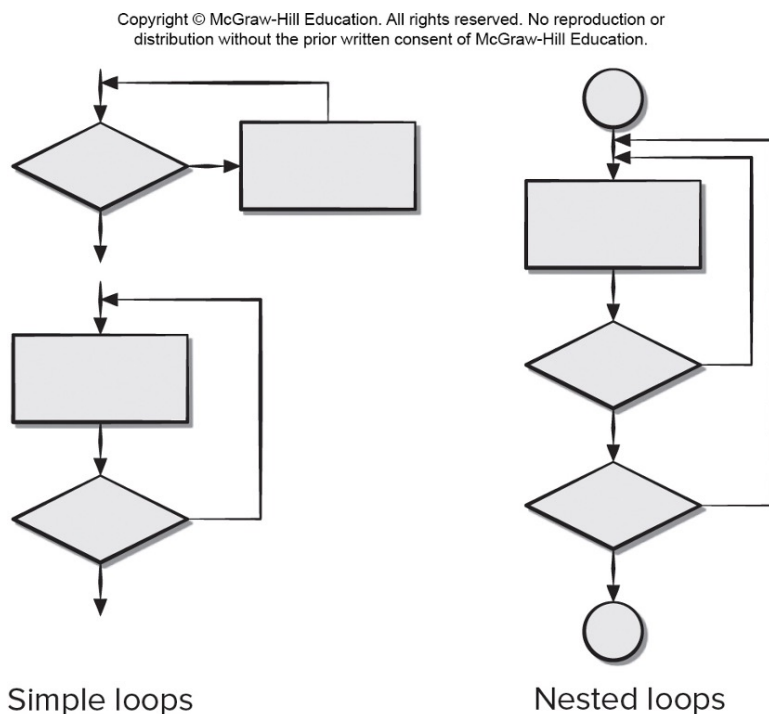
Designing Test Cases

- Using the design or code as a foundation, draw a corresponding flow graph.
- Determine the cyclomatic complexity of the resultant flow graph.
- Determine a basis set of linearly independent paths.
- Prepare test cases that will force execution of each path in the basis set.

Control Structure Testing

- **Condition testing** is a test-case design method that exercises the logical conditions contained in a program module.
- **Data flow testing** selects test paths of a program according to the locations of definitions and uses of variables in the program.
- **Loop testing** is a white-box testing technique that focuses exclusively on the validity of loop constructs.

Classes of Loops



Loop Testing

Test cases for simple loops:

1. Skip the loop entirely.
2. Only one pass through the loop.
3. Two passes through the loop.
4. m passes through the loop where $m < n$.
5. $n - 1, n, n + 1$ passes through the loop.

Test cases for nested loops:

1. Start at the innermost loop. Set all other loops to minimum values.
2. Conduct simple loop tests for the innermost loop while holding the outer loops at their minimum iteration parameter (for example, loop counter) values.
3. Add other tests for out-of-range or excluded values.
4. Work outward, conducting tests for the next loop, but keeping all other outer loops at minimum values and other nested loops to “typical” values.
5. Continue until all loops have been tested.

Black Box Testing

Black-box (functional) testing attempts to find errors in the following categories:

1. Incorrect or missing functions.
2. Interface errors.
3. Errors in data structures or external database access.
4. Behavior or performance errors.
5. Initialization and termination errors.

Unlike white-box testing, which is performed early in the testing process, black-box testing tends to be applied during later stages of testing.

Black-box test cases are created to answer questions like:

- How is functional validity tested?
- How are system behavior and performance tested?
- What classes of input will make good test cases?
- Is the system particularly sensitive to certain input values?
- How are the boundaries of a data class isolated?
- What data rates and data volume can the system tolerate?
- What effect will specific combinations of data have on system operation?

Black Box - Interface Testing

- **Interface testing** is used to check that a program component accepts information passed to it in the proper order and data types and returns

information in proper order and data format.

- Components are **not stand-alone programs** testing interfaces requires the use stubs and drivers.
- Stubs and drivers sometimes incorporate test cases to be passed to the component or accessed by the component.
- Debugging code may need to be inserted inside the component to check that data passed was received correctly.
- Equivalence partitioning - Look into supplemental notes.

Object-Oriented Testing (OOT)

To adequately test OO systems, three things must be done:

- The definition of testing must be broadened to include error discovery techniques applied to object-oriented analysis and design models.
- The strategy for unit and integration testing must change significantly.
- The design of test cases must account for the unique characteristics of OO software.

Black Box - Boundary Value Analysis (BVA) - Functional Testing

- Boundary value analysis leads to a selection of test cases that exercise bounding values.
- Guidelines for B V A:
 1. If an input condition specifies a range bounded by values a and b, test cases should be designed with values a and b and just above and just below a and b.
 2. If an input condition specifies a number of values, test cases should be developed that exercise the min and max numbers as well as values just above and below min and max.
 3. Apply guidelines 1 and 2 to output conditions.
 4. If internal program data structures have prescribed boundaries (for example, array with max index of 100) be certain to design a test case to exercise the data structure at its boundary.

OOT - Class Testing

- Class testing for object-oriented (OO) software is the equivalent of unit testing for conventional software.
- Unlike unit testing of conventional software, which tends to focus on the algorithmic detail of a module and the data that flow across the module interface.
- Class testing for OO software is driven by the operations encapsulated by the class and the state behavior of the class.
- **Valid sequences of operations and their permutations are used to test that class behaviors** - equivalence partitioning can reduce number sequences needed.

Testing Fundamentals

Attributes of a good test:

- A good test has a high probability of finding an error.
- A good test is not redundant.
- A good test should be "best of breed."
- A good test should be neither too simple nor too complex.

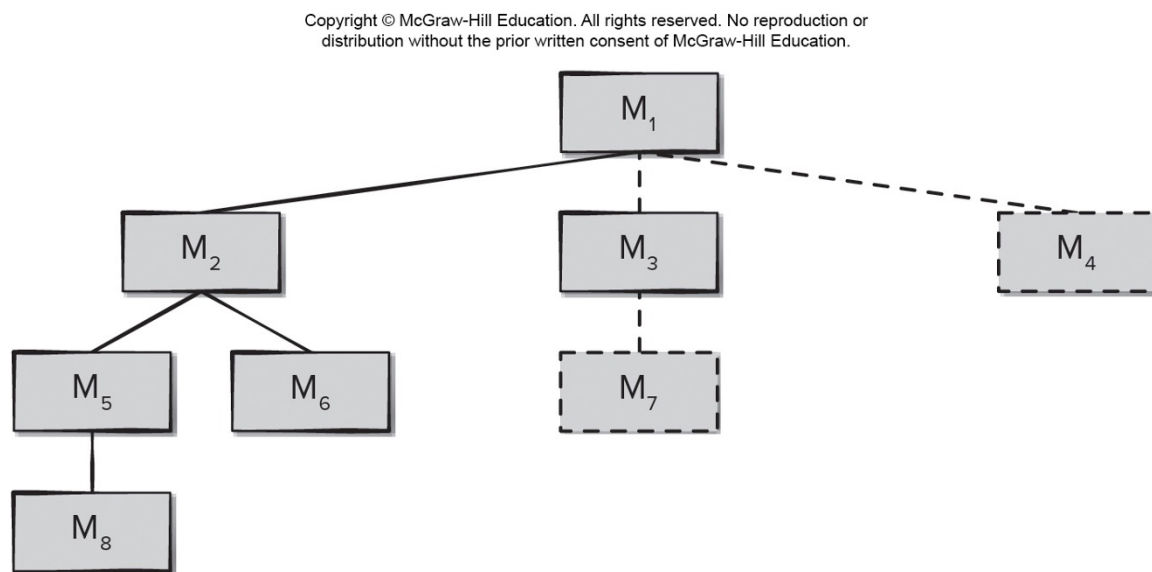
Integration Testing

- **Integration testing** is a systematic technique for constructing the software architecture while conducting tests to uncover errors associated with interfacing.
- The objective is to take unit-tested components and build a program structure that matches the design.
- In the **big bang** approach, all components are combined at once and the entire program is tested as a whole. Chaos usually results!
- In **incremental integration** a program is constructed and tested in small increments, making errors easier to isolate and correct. Far more cost-effective!

Top-Down Integration 1

- **Top-down integration testing** is an incremental approach to construction of the software architecture.
- Modules are integrated by moving downward through the control hierarchy, beginning with the main control module (main program).
- Modules subordinate to the main control module are incorporated into the structure followed by their subordinates.
- **Depth-first integration** integrates all components on a major control path of the program structure before starting another major control path.
- **Breadth-first integration** incorporates all components directly subordinate at each level, moving across the structure horizontally before moving down to the next level of subordinates.

Top-Down Integration 2



Top-Down Integration Testing

1. The main control module is used as a test driver, and stubs are substituted for all components directly subordinate to the main control module.
2. Depending on the integration approach selected (for example, depth or breadth first), subordinate stubs are replaced one at a time with actual components.
3. Tests are conducted as each component is integrated.
4. On completion of each set of tests, another stub is replaced with the real component.

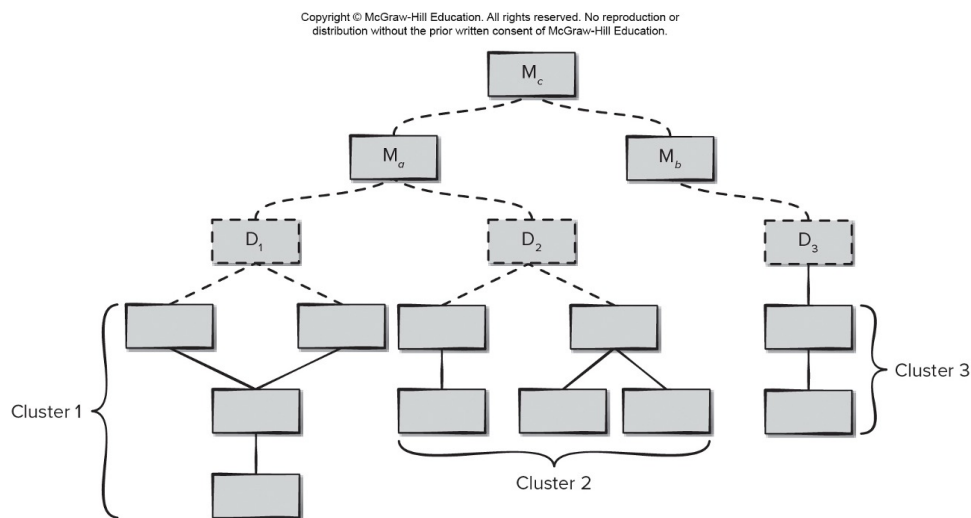
5. Regression testing may be conducted to ensure that new errors have not been introduced.

Bottom-Up Integration Testing

Bottom-up integration testing, begins construction and testing with atomic modules components at the lowest levels in the program structure.

1. Low-level components are combined into clusters (**builds**) that perform a specific software subfunction.
2. A **driver** (a control program for testing) is written to coordinate test-case input and output.
3. The cluster is tested.
4. Drivers are removed and clusters are combined, moving upward in the program structure.

Bottom-Up Integration



Continuous Integration

- **Continuous integration** is the practice of merging components into the evolving software increment at least once a day.
- This is a common practice for teams following agile development practices such as XP or DevOps. Integration testing must take place quickly and efficiently if a team is attempting to always have a working program in place as part of continuous delivery.

- **Smoke testing** is an integration testing approach that can be used when software is developed by an agile team using short increment build times.

Smoke Testing Integration

1. Software components that have been translated into code are integrated into a build. – that includes all data files, libraries, reusable modules, and components required to implement one or more product functions.
2. A series of tests is designed to expose “show-stopper” errors that will keep the build from properly performing its function cause the project to fall behind schedule.
3. The build is integrated (either top-down or bottom-up) with other builds, and the entire product (in its current form) is smoke tested daily.

Smoke Testing Advantages

- **Integration risk is minimized**, since smoke tests are run daily.
- **Quality of the end product is improved**, functional and architectural problems are uncovered early.
- **Error diagnosis and correction are simplified**, errors are most likely in (or caused by) the new build.
- **Progress is easier to assess**, each day more of the final product is complete.
- Smoke testing resembles regression testing by ensuring newly added components do not interfere with the behaviors of existing components.

Regression Testing

- **Regression testing** is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.
- Whenever software is corrected, some aspect of the software configuration (the program, its documentation, or the data that support it) is changed.
- Regression testing helps to ensure that changes (due to testing or for other reasons) do not introduce unintended behavior or additional errors.
- Regression testing may be conducted manually, by re-executing a subset of all test cases or using automated capture/playback tools.

- AI tools may be able to help select the best subset of test cases to use in regression automatically based on previous experiences of the developers with the evolving software product.

Validation Testing

- **Validation testing** tries to uncover errors, but the focus is at the requirements level - on user visible actions and user-recognizable output from the system.
- Validation testing begins at the culmination of integration testing, the software is completely assembled as a package and errors have been corrected.
- Each user story has user-visible attributes, and the customer's acceptance criteria which forms the basis for the test cases used in validation-testing.
- A **deficiency list** is created when a deviation from a specification is uncovered and their resolution is negotiated with all stakeholders.
- An important element of the validation process is a **configuration review** (audit) that ensures the complete system was built properly.

Week 13

Software Release and Maintenance

Release Management

- A system release is a version of a software system that is distributed to customers.
- From deployment to market.
- For mass market software, it is usually possible to identify two types of release: major releases which deliver significant new functionality, and minor releases, which repair bugs and fix customer problems that have been reported. Cost associated with release for major releases but minor may be free.
- For custom software or software product lines, releases of the system may have to be produced for each customer and individual customers may be

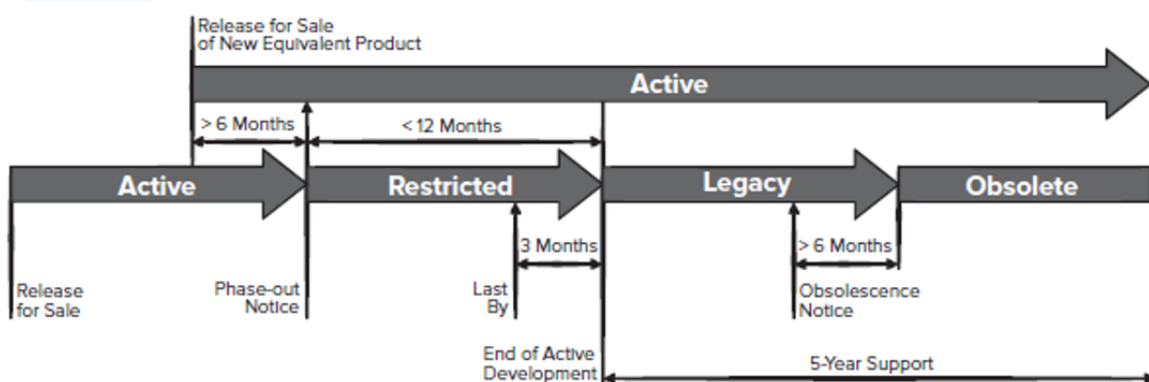
running several different releases of the system at the same time. May have different releases for different people and may consume a lot of time.

Release Components

- As well as the the executable code of the system, a release may also include:
 - configuration files defining how the release should be configured for particular installations;
 - data files, such as files of error messages, that are needed for successful system operation;
 - an installation program that is used to help install the system on target hardware;
 - electronic and paper documentation describing the system; User manual or technical document
 - packaging and associated publicity that have been designed for that release.
 - Handout document/ handover document/ closing document

Software Retirement

FIGURE 27.2 Software release and retirement example



Release Creation

- The executable code of the programs and all associated data files must be identified in the version control system.

- Configuration descriptions may have to be written for different hardware and operating systems.
- Update instructions may have to be written for customers who need to configure their own systems.
- Scripts for the installation program may have to be written.
- Web pages have to be created describing the release, with links to system documentation.
- When all information is available, an executable master image of the software must be prepared and handed over for distribution to customers or sales outlets.

Release Tracking

- In the event of a problem, it may be necessary to reproduce exactly the software that has been delivered to a particular customer. Having records.
- When a system release is produced, it must be documented to ensure that it can be re-created exactly in the future.
- Main documentation needed is technical document and user manual and be customer focused.
- This is particularly important for customized, long-lifetime embedded systems, such as those that control complex machines.
 - Customers may use a single release of these systems for many years and may require specific changes to a particular software system long after its original release date.

Release Reproduction

- To document a release, you have to record the specific versions of the source code components that were used to create the executable code.
- Difference between source and executable code.
- You must keep copies of the source code files, corresponding executables and all data and configuration files. For reproduction
- You should also record the versions of the operating system, libraries, compilers and other tools used to build the software.

Release Planning

- As well as the technical work involved in creating a release distribution, advertising and publicity material have to be prepared and marketing strategies put in place to convince customers to buy the new release of the system.
- Release timing
 - If releases are too frequent or require hardware upgrades, customers may not move to the new release, especially if they have to pay for it.
 - If system releases are too infrequent, market share may be lost as customers move to alternative systems.

Software as a Service

- Delivering software as a service (SaaS) reduces the problems of release management.
- It simplifies both release management and system installation for customers.
- The software developer is responsible for replacing the existing release of a system with a new release and this is made available to all customers at the same time.

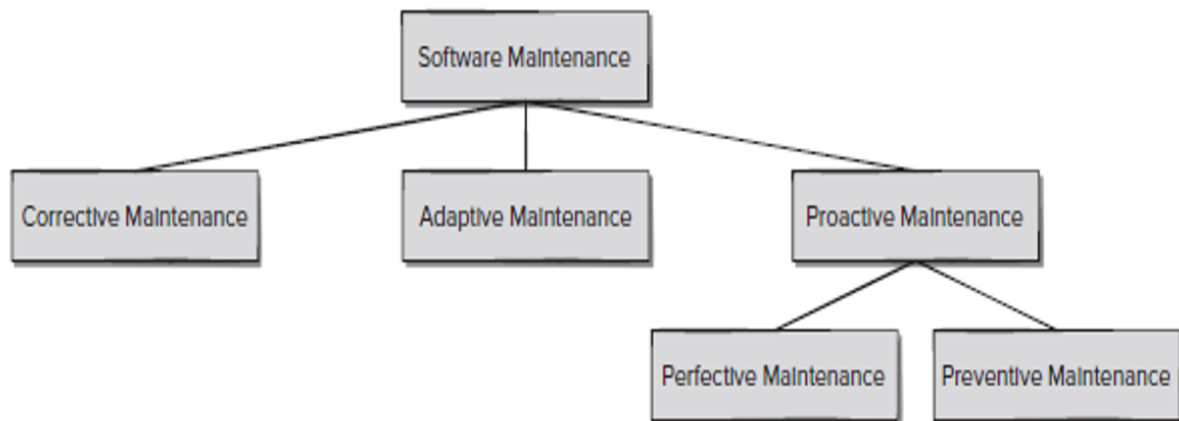
Software Maintenance

Software Maintenance and support the system

- Modifying a program after it has been put into use. Activates after software is released
- The term is mostly used for changing custom software. Generic software products are said to evolve to create new versions.
- Maintenance does not normally involve major changes to the system's architecture.
- Changes are implemented by modifying existing components and adding new components to the system.
- Not much documentation

Types of Software Maintenance

FIGURE 27.4 Types of software maintenance



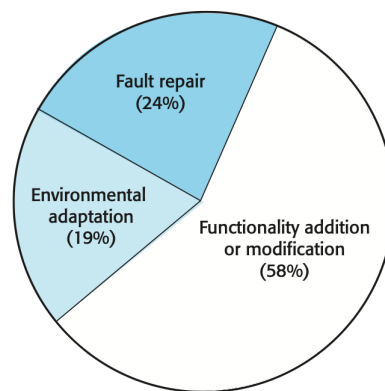
- Reactive Maintenance
 - Corrective
 - Adaptive
- Proactive
 - Perfective
 - Preventive

Types of Maintenance

- Fault repairs - Corrective maintenance
 - Changing a system to fix bugs/vulnerabilities and correct deficiencies in the way meets its requirements.
- Environmental adaptation – Adaptive maintenance
 - Maintenance to adapt software to a different operating environment
 - Changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation.
- Functionality addition and modification – perfective
 - Modifying the system to satisfy new requirements.

Maintenance Effort Distribution

- 21% corretive
- 25%
Adaptive
- 4%
presentive
- 50%
perfective



Maintenance Costs

- Usually greater than development costs (2* to 100* depending on the application).
- Affected by both technical and non-technical factors.
- Increases as software is maintained. Maintenance corrupts the software structure so makes further maintenance more difficult.
- Aging software can have high support costs (e.g. old languages, compilers etc.).
- Aging software is also known as legacy software
- Maintenance is given new developers
- It is usually more expensive to add new features to a system during maintenance than it is to add the same features during development
 - A new team has to understand the programs being maintained
 - Separating maintenance and development means there is no incentive for the development team to write maintainable software
 - Program maintenance work is unpopular
 - Maintenance staff are often inexperienced and have limited domain knowledge.
 - As programs age, their structure degrades and they become harder to change

Process Metrics

- Process metrics may be used to assess maintainability
 - Number of requests for corrective maintenance;
 - Average time required for impact analysis;
 - Average time taken to implement a change request;
 - Number of outstanding change requests.
- If any or all of these is increasing, this may indicate a decline in maintainability. May be caused by preexisting issues or lack of documentation from past versions

Reverse Engineering

- The first task that needs to be completed by software engineers before responding to any maintenance request is to understand the system that needs to be modified. Sadly, the system being maintained often has low quality and lacks reasonable documentation. This is what technical debt is all about.
- //Technical debt is often caused by developers adding features without documenting them or considering their impact on the larger software system.
- Reverse engineering can be used to extract design information from source code, but the abstraction level of this information, the completeness of the documentation, and the degree to which human analysts work comfortably with the available tools, are highly variable.
- Reverse engineering requires developers to evaluate the old software system by examining its (often undocumented) source code, developing a meaningful specification of the processing being performed, the user interface that was used, and the program data structures or associated database. Also also utilize documentation but isn't needed

Software Reengineering

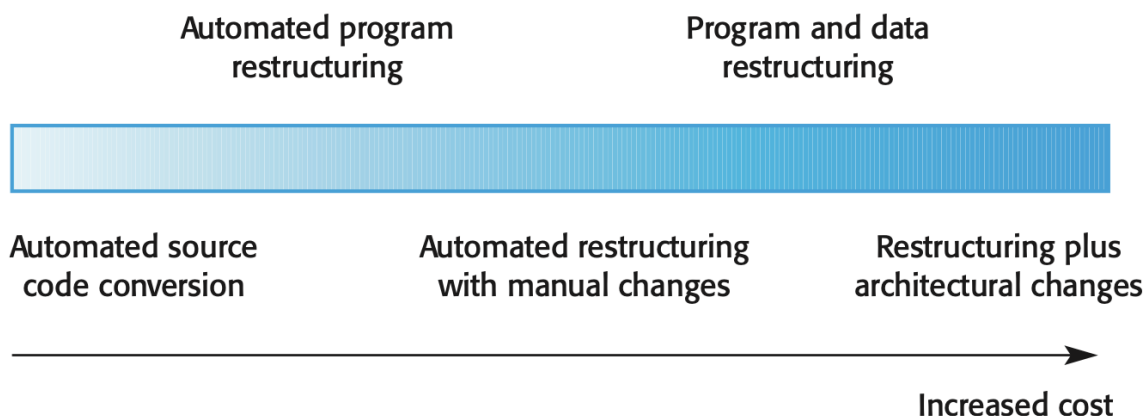
- Restructuring or rewriting part or all would be for of a legacy system without changing its functionality.
- Applicable where some but not all sub-systems of a larger system require frequent maintenance.

- Reengineering involves adding effort to make them easier to maintain. The system may be re-structured and re-documented.

Advantages of reengineering

- Reduced risk
 - There is a high risk in new software development. There may be development problems, staffing problems and specification problems.
- Reduced cost
 - The cost of re-engineering is often significantly less than the costs of developing new software.

Reengineering Approaches



Reengineering Cost Factors

- The quality of the software to be reengineered.
- The tool support available for reengineering.
- The extent of the data conversion which is required.
- The availability of expert staff for reengineering.
 - This can be a problem with old systems based on technology that is no longer widely used.

Refactoring

- Refactoring is the process of making improvements to a program to slow down degradation through change.

- You can think of refactoring as 'preventative maintenance' that reduces the problems of future change.
- Refactoring involves modifying a program to improve its structure, reduce its complexity or make it easier to understand.
- When you refactor a program, you should not add functionality but rather concentrate on program improvement.

Refactoring and Reengineering

- Re-engineering takes place after a system has been maintained for some time and maintenance costs are increasing. You use automated tools to process and re-engineer a legacy system to create a new system that is more maintainable.
- Refactoring is a continuous process of improvement throughout the development and evolution process. It is intended to avoid the structure and code degradation that increases the costs and difficulties of maintaining a system. XP uses refactoring and pair programming. During the design phase for XP.

'Bad Smells' in program code

- Duplicate code
 - The same or very similar code may be included at different places in a program. This can be removed and implemented as a single method or function that is called as required.
- Long methods
 - If a method is too long, it should be redesigned as a number of shorter methods.
- Switch (case) statements
 - These often involve duplication, where the switch depends on the type of a value. The switch statements may be scattered around a program. In object-oriented languages, you can often use polymorphism to achieve the same thing.
- Data clumping
 - Data clumps occur when the same group of data items (fields in classes, parameters in methods) re-occur in several places in a

program. These can often be replaced with an object that encapsulates all of the data.

- Speculative generality
 - This occurs when developers include generality in a program in case it is required in the future. This can often simply be removed.